1
Getting Started

We kick off this book with a collection of relatively elementary programming problems, none of which require ideas more advanced than arrays and iteration.

Elementary does not necessarily mean easy, however! These problems provide a good introduction to the demanding nature of the robot judge, and the need to read carefully and understand specifications. They also provide an opportunity to discuss programming styles best suited to getting the job done.

To help you get started, we begin with a description of the robot judges and their idiosyncrasies. We follow with a discussion of basic programming style and data structures before introducing our first set of problems. As in all chapters in this book, we follow with hints for selected problems and notes for further study.

1.1 Getting Started With the Judge

This book is designed to be used in tandem with one (or both) of two robot judging websites. The Programming Challenges judge *http://www.programming-challenges.com* has been set up specifically to help you get the most from the challenges in this book. The Universidad de Valladolid judge *http://online-judge.uva.es* has a different interface as well as hundreds of additional problems available.

All the problems in the book can be judged from either website, which are both administered by Miguel Revilla. In this section, we describe how to use the two judges and explain the differences between them. Be aware that both sites are living, breathing projects, so these procedures may evolve over time. Check the current instructions at each site for clarification.

Your first task is to get an account for the judge of your choice. You will be asked to give a password governing access to your personal data, specifically your name and your email address. If you forget your password, clicking the appropriate button will get it emailed back to you.

Note that the contestant rosters of the two sites are currently kept distinct, but there is no reason why you should not register for both of them and enjoy their distinct advantages.

1.1.1 The Programming Challenges Robot Judge

The Programming Challenges website (*http://www.programming-challenges.com*) provides special features associated with each of the problems in this book. For example, a description of each challenge appearing in the book is given on site, along with down-loadable input and output files to eliminate the need for you to type this test data.

The Programming Challenges site uses a web interface for submission (the *Submit-o-Matic*) instead of the email interface of the UVa judge. This makes submission much easier and more reliable, and provides for quicker response.

Each problem in this book has two associated ID numbers, one for each judge. One advantage of the web interface is that the identifier for the Programming Challenges site (the PC ID) is not necessary for most submissions. The problem descriptions in this book have been rewritten for clarity; thus they often differ from the descriptions on the UVa judge in minor ways. However, the problems they describe are identical. Thus any solution scored as correct on one judge *should be* scored correct on the other as well.

The Programming Challenges site has a special course management interface, which permits an instructor to maintain a roster of students in each class and see their submissions and results. It also contains a program similarity tester so the instructor can verify that the solutions each student submits are indeed his or her own work. This makes it "bad karma" to hunt for solutions on the web or in your classmate's directories.

1.1.2 The Universidad de Valladolid Robot Judge

All of the problems in this book and many more appear on the Universidad de Valladolid robot judge *http://online-judge.uva.es*, the largest collection of programming problems in the world. We encourage anyone whose appetite has been whetted by our challenges to continue their studies there.

After registering on the UVa judge, you will receive email containing an ID number which will uniquely identify your programs to the judge. You will need this ID number for every solution you submit.

The UVa judge is gradually adopting a web interface but currently uses email submission. Solutions are emailed directly to *judge@uva.es* after being annotated with enough information to tell the judge which problem you are trying to solve, who the author is, and what programming language you are using.

Specifically, each submitted program must contain a line (at any location) with an `@JUDGE_ID:` field. Usually, this line is placed inside a comment. For example,

```
/*   @JUDGE_ID:   1000AA   100   C   "Dynamic Programming"   */
```

The argument after the `@JUDGE_ID:` is your user ID (`1000AA` in the example). This is followed by the problem number (100 in the example), and then by the language used. *Make sure you use the UVa ID number for all submissions to this judge!* Upper- and lowercase letters are indistinguishable. If you fail to specify the programming language, the judge will try to auto-detect it – but why play games? Finally, if you have used any interesting algorithm or method, you may include a note to that effect between quotes, such as `Dynamic Programming` in the example above.

Bracketing your program with beginning/end of source comments is a good way to make sure the judge is not confused by junk appended by your mailer.

```
/* @BEGIN_OF_SOURCE_CODE */

your program here

/* @END_OF_SOURCE_CODE */
```

Certain mysterious errors will go away when you do this.

1.1.3 Feedback From the Judge

Students should be aware that both judges are often very picky as to what denotes a correct solution. It is very important to interpret the problem specifications properly. *Never* make an assumption which is not explicitly stated in the specs. There is absolutely no reason to believe that the input is sorted, the graphs are connected, or that the integers used in a problem are positive and reasonably small unless it says so in the specification.

Just like the human judges of the ACM International Collegiate Programming Contest, the online judge provides you with very little feedback about what is wrong with your program. The judge is likely to return one of the following verdicts:

- *Accepted (AC)* — Congratulations! Your program is correct, and runs within the given time and memory limits.
- *Presentation Error (PE)* — Your program outputs are correct but are not presented in the specified format. Check for spaces, left/right justification, line feeds, etc.
- *Accepted (PE)* — Your program has a minor presentation error, but the judge is letting you off with a warning. Don't be concerned, because many problems have somewhat ambiguous output specifications. Usually your problems are something as trivial as an extra blank at the end of a line, so stop here and declare victory.

- *Wrong Answer (WA)* — This you should concern you, because your program returned an incorrect answer to one or more of the judge's secret test cases. You have some more debugging to do.
- *Compile Error (CE)* — The compiler could not figure out how to compile your program. The resulting compiler messages will be returned to you. Warning messages that do not interfere with compilation are ignored by the judge.
- *Runtime Error (RE)* — Your program failed during execution due to a segmentation fault, floating point exception, or similar problem. Its dying message will be sent back to you. Check for invalid pointer references or division by zero.
- *Time Limit Exceeded (TL)* — Your program took too much time on at least one of the test cases, so you likely have a problem with efficiency. Just because you ran out of time on one input does not mean you were correct on all the others, however!
- *Memory Limit Exceeded (ML)* — Your program tried to use more memory than the judge's default settings.
- *Output Limit Exceeded (OL)* — Your program tried to print too much output. This usually means it is trapped in a infinite loop.
- *Restricted Function (RF)* — Your source program tried to use an illegal system function such as `fork()` or `fopen()`. Behave yourself.
- *Submission Error (SE)* — You did not correctly specify one or more of the information fields, perhaps giving an incorrect user ID or problem number.

Just to reiterate: if your program is found guilty of having a wrong answer, the judge will not show you which test case it failed on, or give you any additional hints as to why it failed. This is why it is so essential to review the specifications carefully. Even when you may be *sure* that your program is correct, the judge may keep saying no. Perhaps you are overlooking a boundary case or assuming something which just ain't so. Resubmitting the program without change does you absolutely no good. Read the problem again to make sure it says what you thought it did.

The judge occasionally returns a more exotic verdict which is essentially independent of your solution. See the appropriate website for details.

1.2 Choosing Your Weapon

What programming language should you use in your battles with the judge? Most likely, the language which you know best. The judge currently accepts programs written in C, C++, Pascal, and Java, so your favorite language is probably available. One programming language may well be better than another for a specific programming task. However, these problems test general problem-solving skills far more than portability, modularity, or efficiency, which are the usual dimensions by which languages are compared.

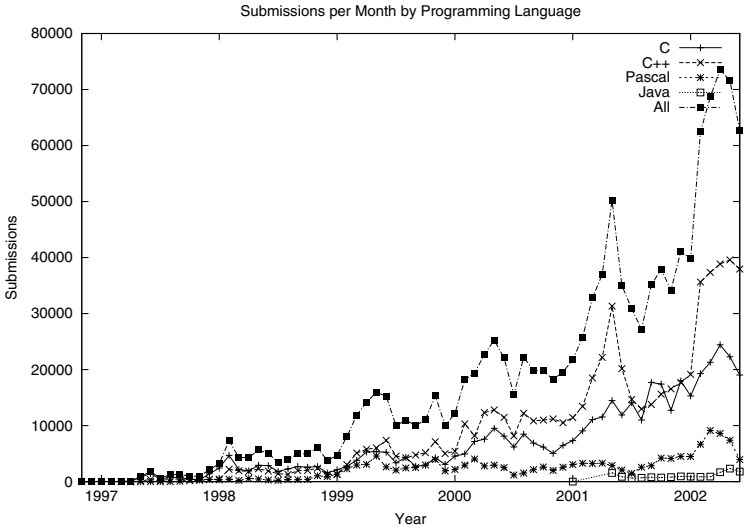


Figure 1.1. Robot Judge Submissions by Programming Language Through December 2002.

1.2.1 Programming Languages

The four languages supported by the judge were designed at different times with different goals in mind:

- *Pascal* — The most popular educational programming language of the 1980s, Pascal was designed to encourage good structured-programming habits. Its popularity has eroded almost to the point of extinction, but it retains a foothold in high schools and in Eastern Europe.

- *C* — The original language of the UNIX operating system, C was designed to provide experienced programmers with the power to do whatever needs to be done. This includes the power to hang yourself by invalid pointer references and invalid type casting. Developments in object-oriented programming during the 1990s lead to the new and improved...

- *C++* — The first commercially successful object-oriented language pulled off the neat trick of maintaining backward compatibility with C while incorporating new data abstraction and inheritance mechanisms. C++ became the primary programming language for teaching and industry during the mid-to-late 1990s, but now it looks over its shoulder at...

- *Java* — Designed as a language to support mobile programs, Java has special security mechanisms to avoid common programmer errors such as array out-of-bounds violations and illegal pointer access. It is a full-featured programming language which can do everything the others can and more.

Lang	Total	AC	PE	WA	CE	RE	TL	ML	OL	RF
C	451447	31.9%	6.7%	35.4%	8.6%	9.1%	6.2%	0.4%	1.1%	0.6%
C++	639565	28.9%	6.3%	36.8%	9.6%	9.0%	7.1%	0.6%	1.0%	0.7%
Java	16373	17.9%	3.6%	36.2%	29.8%	0.5%	8.5%	1.0%	0.5%	2.0%
Pascal	149408	27.8%	5.5%	41.8%	10.1%	6.2%	7.2%	0.4%	0.4%	0.5%
All	1256793	29.7%	6.3%	36.9%	9.6%	8.6%	6.8%	0.5%	1.0%	0.6%

Table 1.1. The Judge's Verdicts by Programming Language (Through December 2002).

Note that each of the judge's programming languages have compiler and system-specific idiosyncrasies. Thus a program which runs on your machine may not run on the judge. Read the language notes on your judge's website carefully to minimize trouble, particularly if you are using Java.

It is interesting to look at which languages people have been using. As of December 2002 over 1,250,000 program submissions have been sent to the robot judge. Almost half of them were in C++, with almost another third in C. Only a tiny fraction were written in Java, but that is not a fair test since the judge did not accept Java programs until November 2001.

These submissions are broken down by month in Figure 1.1. C proved the most popular language until late 1999, when C++ surged ahead. It is interesting to note the annual spike in demand each fall as students train for the ACM International Collegiate Programming Contest regional competitions. Every year the judge gets busier, as more and more students seek trial in its court.

It is also interesting to look at the judge's verdicts by programming language. These are tabulated in Table 1.1, according to the response codes described in Section 1.1.3. The verdicts are quite consistent across the board. However, the frequencies of certain types of errors appear to be language dependent. C++ programs run out of time and memory more often than C language programs, a sign that C++ is a relative resource hog. C has a slightly higher acceptance rate than C++, presumably reflecting its popularity at an earlier state in the judge's development. Pascal has the lowest rate of restricted function errors, reflecting its origins as a nice, safe language for students to play with. Java has had *far* more than its share of compiler errors to date, but it also crashes much less often than the other languages. Safety is indeed a virtue.

But the basic lesson is that the tools do not make the man. Your language doesn't solve the problems – you do.

1.2.2 Reading Our Programs

Several programming examples appear in this book, illustrating programming techniques and providing complete implementations of fundamental algorithms. All of this code is available at *http://www.programming-challenges.com* for you to use and experiment with. There is no finer way to debug programs than having them read by several thousand bright students, so look there for errata and revised solutions.

Our programming examples are implemented in a vanilla subset of C, which we hope will be understandable by all of our readers with relatively little effort. C itself is a

subset of C++ and its syntax is quite similar to Java. We have taken care to avoid using weird C-specific constructs, pointer structures and dynamic memory allocation throughout this book, so what remain should be familiar to users of all four of the judge's programming languages.

We provide a few hints about C below which may be helpful in reading our programs:

- *Parameter Passing* — All parameters in C are passed by call-by-value, meaning that copies of all arguments are made on function calls. This would *seem* to suggest that it is impossible to write functions that have side effects. Instead, C encourages you to pass a pointer to any argument that you intend to modify within the body of the function.

  Our only use of pointers will be in parameter passing. The pointer to `x` is denoted by `&x`, while the item pointed to by `p` is denoted `*p`. Do not get confused between multiplication and pointer dereferencing!

- *Data Types* — C supports several basic data types, including `int`, `float`, and `char`, which should all be self-explanatory. Higher precision `int`s and `float`s are denoted `long` and `double`, respectively. All functions return a value of type `int` if not otherwise specified.

- *Arrays* — C array indices always range from 0 to $n-1$, where n is the number of elements in the array. Thus if we want to start with a first index of 1 for convenience, we had better remember to allocate room for $n+1$ elements in the array. No runtime checking is performed on the validity of array bounds, so such errors are a common cause of program crashes.

  We are not always consistent as to where the first element of each array is located. Starting from 0 is the traditional C style. However, it is sometimes clearer or easier to start at 1, and we are willing to pay one extra memory location for the privilege. Try not to be confused when reading our code.

- *Operators* — C contains a few essential operators which may be mysterious to some readers. The integer remainder or *mod* operation is denoted `%`. The logical-and and logical-or operations which appear in conditional statements are denoted `&&` and `||`, respectively.

1.2.3 Standard Input/Output

UNIX programmers are familiar with notions of filters and pipes–programs that take one stream of input and produce one stream of output. The output of such programs is suitable to feed to other programs as input. The paradigm is one of stringing lots of little programs together rather than producing big, complicated software systems that try to do everything.

This software tools philosophy has taken somewhat of a beating in recent years due to the popularity of graphical user interfaces. Many programmers instinctively put a point-and-click interface on every program. But such interfaces can make it very difficult to transfer data from one program to another. It is easy to manipulate text output in another program, but what can you do with an image other than look at it?

```c
#include<stdio.h>

int main() {
   long p,q,r;

   while (scanf("%ld %ld",&p,&q)
               !=EOF) {
     if (q>p) r=q-p;
     else r=p-q;

     printf("%ld\n",r);
   }
}
```

```cpp
#include<iostream.h>

void main()
{
long long a,b,c;

while (cin>>a>>b) {
   if (b>a)
      c=b-a;
   else
      c=a-b;
   cout << c << endl;
}
}
```

```
{$N+}
program acm;
var
  a, b, c : integer;
begin
  while not eof do
  begin
    readln(a, b);
    if b > a then
    begin
      c := b;
      b := a;
      a := c
    end;
    writeln(a - b);
  end
end.
```

Figure 1.2. Standard Input/Output Examples in C (left), C++ (center), and Pascal (right).

The judge's I/O standards reflect the official ACM programming contest rules. Each program must read the test data from the standard input and print the results to the standard output. Programs are not allowed to open files or to execute certain system calls.

Standard input/output is fairly easy in C, C++, and Pascal. Figure 1.2 provides a simple example in each language that reads in two numbers per line and reports the absolute value of their difference. Note how your favorite language tests for the end-of-file terminating condition. Most problems make input processing even easier by specifying a count of the number of examples or describing a special termination line.

Most languages provide powerful formatted I/O functions. When used properly, single-line commands can render unnecessary certain painful parsing and formatting routines written by those who didn't read the manual.

Standard input/output is *not* easy in Java, however. A electronic template for Java I/O (35 lines long) is available from *http://www.programming-challenges.com*. Set it up once and use it for all your entries.

Java programs submitted to the judge must consist of a single source code file. They are currently compiled and run as native applications using the `gcj` compiler, although this may change in the future. Note that `java::io` use is restricted; which implies that some features are not available. Network functions and threads are also unavailable. However, methods from `math`, `util` and other common packages are authorized. All programs must begin in a static main method in a `Main` class. Do not use public classes: even `Main` must be non-public to avoid compile errors. However, you can add and instance as many classes as needed.

If you find yourself using an operating system/compiler which makes it difficult to use standard input/output, note that the judge always defines the `ONLINE_JUDGE` symbol while compiling your program. Thus, you can test for it and redirect the input/output to a file when running on your own system.

1.3 Programming Hints

It is not our purpose in this book to teach you how to program, only how to program better. We assume you are familiar with such fundamental concepts as variables, conditional statements (e.g., `if-then-else`, `case`), iteration primitives (e.g., `for-do`, `while-do`, `repeat-until`), subroutines, and functions. If you are unfamiliar with these concepts you may have picked up the wrong book, but buy it anyway for later use.

It is important to realize how much power there is in what you already know. In principle, every interesting algorithm/program can be built from what you learn in a first programming course. The powerful features of modern programming languages are not really necessary to build interesting things – only to do them in cleaner, better ways.

To put it another way, one becomes a good writer not by learning additional vocabulary words but by finding something to say. After one or two programming courses, you know all the words you need to make yourself understood. The problems in this book strive to give you something interesting to say.

We offer a few low-level coding hints that are helpful in building quality programs. The bad examples all come from actual submissions to the robot judge.

- *Write the Comments First* — Start your programs and procedures by writing a few sentences explaining what they are supposed to do. This is important because if you can't *easily* write these comments, you probably don't really understand what the program does. We find it much easier to debug our comments than our programs, and believe the additional typing is time very well spent. Of course, with the time pressure of a contest comes a tendency to get sloppy, but do so at your own risk.

- *Document Each Variable* — Write a one-line comment for each variable when you declare it so you know what it does. Again, if you can't describe it easily, you don't know why it is there. You will likely be living with the program for at least a few debug cycles, and this is a modest investment in readability which you will come to appreciate.

- *Use Symbolic Constants* — Whenever you have a constant in your program (input size, mathematical constant, data structure size, etc.) declare it to be so at the top of your program. Horribly insidious errors can result from using inconsistent constants in a program. Of course, the symbolic name helps only if you actually use it in your program whenever you need the constant...

- *Use Enumerated Types for a Reason* — Enumerated types (i.e., symbolic variables such Booleans (`true`,`false`)) can be terrific aids to understanding. However, they are often unnecessary in short programs. Note this example representing the suit (club, diamond, heart, spade) of a deck of cards:

```
switch(cursuit) {
    case 'C':
      newcard.suit = C;
```

```
      break;
    case 'D':
      newcard.suit = D;
      break;
    case 'H':
      newcard.suit = H;
      break;
    case 'S':
      newcard.suit = S;
...
```

No additional clarity arises from using the enumerated variables `(C,D,H,S)` over the original character representation `('C','D','H','S')`, only additional opportunities for error.

- *Use Subroutines To Avoid Redundant Code* — Read the following program fragment managing the state of a rectangular board, and think how you might shorten and simplify it:

```
...

     while (c != '0') {
        scanf("%c", &c);
        if (c == 'A') {
            if (row-1 >= 0) {
                temp = b[row-1][col];
                b[row-1][col] = ' ';
                b[row][col] = temp;
                row = row-1;
            }
        }
        else if (c == 'B') {
            if (row+1 <= BOARDSIZE-1) {
                temp = b[row+1][col];
                b[row+1][col] = ' ';
                b[row][col] = temp;
                row = row+1;
            }
        }
...
```

In the full program, there were four blocks of three lines each moving a value to a neighboring cell. Mistyping a single + or − would have lethal consequences. Much safer would be to write a single move-swap routine and call it with the proper arguments.

- *Make Your Debugging Statements Meaningful* — Learn to use the debugging environment on your system. This will enable you to stop execution at a given statement or condition, so you can see what the values of all associated variables are. This is usually faster and easier than typing in a bunch of print statements. But if you *are* going to insert debugging print statements, make them say something. Print out all relevant variables, and *label* the printed quantity with the variable name. Otherwise it is easy to get lost in your own debugging output.

Most computer science students are now well-versed in *object-oriented programming*, a software engineering philosophy designed to construct reusable software components and exploit them. Object-oriented programming is useful to build large, reusable programs. However, most of the programming challenge problems in this book are designed to be solved by short, clever programs. The basic assumption of object-oriented programming just does not apply in this domain, so defining complicated new objects (as opposed to *using* predefined objects) is likely to be a waste of time.

The trick to successful programming is not abandoning style, but using one appropriate to the scale of the job.

1.4 Elementary Data Types

Simple data structures such as arrays have an important advantage over more sophisticated data structures such as linked lists: they are simple. Many kinds of errors in pointer-based structures simply cannot happen with static arrays.

The sign of a mature professional is keeping the simple jobs simple. This is particularly challenging for those who are just learning a new subject. Medical students provide a classic example of this problem. After sitting through a few lectures on obscure tropical diseases, a young doctor worries that any patient with a sniffle and a rash might have the Ebola virus or bubonic plague, while a more experienced physician just sends them home with a bottle of aspirin.

Likewise, you may have recently learned about balanced binary search trees, exception handling, parallel processing, and various models of object inheritance. These are all useful and important subjects. But they are not necessarily the best way to get a correct program working for a simple job.

So, yes, pointer-based structures are very powerful if you do not know the maximum size of the problem in advance, or in supporting fast search and update operations. However, many of the problems you will be solving here have maximum sizes specified. Further, the robot judge typically allows several seconds for your job to complete, which is a lot of computation time when you stop to think about it. You don't get extra points for finishing faster.

So what is the simple, mature approach to data structures? First, be familiar with the basic primitive data *types* built into your programming language. In principle, you can build just about anything you want from just these:

- *Arrays* — This workhorse data type permits access to data by position, not content, just like the house numbers on a street permit access by address, not

name. They are used to store sequences of single-type elements such as integers, reals, or compound objects such as records. Arrays of characters can be used to represent text strings, while arrays of text strings can be used to represent just about anything.

Sentinels can be a useful technique to simplify array-based programming. A sentinel is a guard element, implicitly checking that the program does not run beyond the bounds of the array without performing an explicit test. Consider the case of inserting element x into the proper position among n elements in a sorted array a. We can explicitly test each step to see whether we have hit the bottom of the array as on the left:

```
i = n;
while ((a[i]>=x) && (i>=1)) {
        a[i] = a[i-1];
        i=i-1;
}
a[i+1] = x;
```

```
i = n;
a[0] = - MAXINT;
while (a[i] >= x) {
        a[i] = a[i-1];
        i=i-1;
}
a[i+1] = x;
```

or, we can make sure that fake element `a[0]` is smaller than anything it will encounter as on the right. Proper use of sentinels, and making sure that your array is a little larger than it presumably needs to be, can help avoid many boundary errors.

- *Multidimensional Arrays* — Rectangular grid structures such as chessboards and images comes to mind first when thinking about two-dimensional arrays, but more generally they can be used to group together *homogeneous* data records. For example, an array of n points in the $x - y$ plane can be thought of as an $n \times 2$ array, where the second argument (0 or 1) of $A[i][j]$ dictates whether we are referring to the x or y coordinate of the point.

- *Records* — These are used to group together *heterogeneous* data records. For example, an array of people-records can lump together people's names, ID numbers, heights, and weights into a simple package. Records are important for conceptual clarity in large programs, but such fields can often be harmlessly represented using separate arrays in programs of modest size.

  Whether it is better to use records or multidimensional arrays in a problem is not always a clear-cut decision. Think of the problem of representing points in the $x - y$ plane discussed above. The obvious representation would be a record or structure such as:

```
struct point {
     int x, y;
};
```

instead of as a two-element array. A big plus for records is that the notation `p.y` and `p.y` are more akin to our natural notation for working with points. However, a

disadvantage of the record representation is that you cannot loop over individual variables as you can elements in an array.

Suppose you wanted to change a geometric program to work with three-dimensional points instead of two, or even in arbitrary dimensions. Sure you can easily add extra fields to the record, but every place where you did calculations on x and y you now must replicate them for z. But by using the array representation, changing distance computations from two to three dimensions can be as simple as changing a constant:

```
typedef int point[DIMENSION];

double distance(point a, point b)
{
        int i;
        double d=0.0;

        for (i=0; i<DIMENSION; i++)
                d = d + (a[i]-b[i]) * (a[i]-b[i]);

        return( sqrt(d) );
}
```

In Chapter 2, we will review more advanced data *structures* which can be built from these basic primitives. These will let us work with higher levels of abstraction, but don't be afraid to use simple technology when it suffices for the job.

1.5 About the Problems

Each chapter of this book ends with an appropriate set of programming challenge problems. These challenges have been carefully selected from over 1,000 such problems collected at the Universidad de Valladolid website. We aimed for clear, clever problems with varying degrees of difficulty. We especially looked for that spark of insight which turns a problem into a challenge.

The description of each selected problem has been edited for correctness and readability. We have tried to preserve the local color and flavor of each original problem while making the language reasonably consistent. The identification number for each problem on both judging sites is provided. These numbers are important for successful submission. The first number for each pair is associated with *http://www.programming-challenges.com*; the second number works on *http://online-judge.uva.es*.

To give some idea as to the relative difficulty of the problems, each has been annotated in three different ways. First, each problem has been given a grade of `A`, `B`, or `C`, reflecting how many correct solutions the judge has seen over the years. `A` problems are presumably easier to solve or somehow more attractive than `B` problems. Second, the frequency that submitted solutions for each problem are accepted by the judge is graded as `high`,

`average`, or `low`. Low percentages may indicate particularly finicky judges, or perhaps problems that require more subtlety than is initially apparent. Or they may just reflect bugs in test cases which have since been fixed. Therefore, do not obsess too much about these ratings. Finally, we give a subjective rating (from 1 to 4) of the academic level required to solve the problem. Higher numbers indicate more sophisticated problems.

Good luck, and happy hacking!

1.6 Problems

1.6.1 The $3n + 1$ Problem

PC/UVa IDs: 110101/100, **Popularity:** A, **Success rate:** low **Level:** 1

Consider the following algorithm to generate a sequence of numbers. Start with an integer n. If n is even, divide by 2. If n is odd, multiply by 3 and add 1. Repeat this process with the new value of n, terminating when $n = 1$. For example, the following sequence of numbers will be generated for $n = 22$:

22 11 34 17 52 26 13 40 20 10 5 16 8 4 2 1

It is *conjectured* (but not yet proven) that this algorithm will terminate at $n = 1$ for every integer n. Still, the conjecture holds for all integers up to at least $1,000,000$.

For an input n, the *cycle-length* of n is the number of numbers generated up to and *including* the 1. In the example above, the cycle length of 22 is 16. Given any two numbers i and j, you are to determine the maximum cycle length over all numbers between i and j, *including* both endpoints.

Input

The input will consist of a series of pairs of integers i and j, one pair of integers per line. All integers will be less than 1,000,000 and greater than 0.

Output

For each pair of input integers i and j, output i, j in the same order in which they appeared in the input and then the maximum cycle length for integers between and including i and j. These three numbers should be separated by one space, with all three numbers on one line and with one line of output for each line of input.

Sample Input

```
1 10
100 200
201 210
900 1000
```

Sample Output

```
1 10 20
100 200 125
201 210 89
900 1000 174
```

1.6.2 Minesweeper

PC/UVa IDs: 110102/10189, **Popularity:** A, **Success rate:** high **Level:** 1

Have you ever played Minesweeper? This cute little game comes with a certain operating system whose name we can't remember. The goal of the game is to find where all the mines are located within a $M \times N$ field.

The game shows a number in a square which tells you how many mines there are adjacent to that square. Each square has at most eight adjacent squares. The 4×4 field on the left contains two mines, each represented by a "`*`" character. If we represent the same field by the hint numbers described above, we end up with the field on the right:

```
*...        *100
....        2210
.*..        1*10
....        1110
```

Input

The input will consist of an arbitrary number of fields. The first line of each field contains two integers n and m ($0 < n, m \leq 100$) which stand for the number of lines and columns of the field, respectively. Each of the next n lines contains exactly m characters, representing the field.

Safe squares are denoted by "`.`" and mine squares by "`*`," both without the quotes. The first field line where $n = m = 0$ represents the end of input and should not be processed.

Output

For each field, print the message `Field #`x`:` on a line alone, where x stands for the number of the field starting from 1. The next n lines should contain the field with the "`.`" characters replaced by the number of mines adjacent to that square. There must be an empty line between field outputs.

Sample Input

```
4 4
*...
....
.*..
....
3 5
**...
.....
.*...
0 0
```

Sample Output

```
Field #1:
*100
2210
1*10
1110

Field #2:
**100
33200
1*100
```

1.6.3 The Trip

PC/UVa IDs: 110103/10137, **Popularity:** B, **Success rate:** average **Level:** 1

A group of students are members of a club that travels annually to different locations. Their destinations in the past have included Indianapolis, Phoenix, Nashville, Philadelphia, San Jose, and Atlanta. This spring they are planning a trip to Eindhoven.

The group agrees in advance to share expenses equally, but it is not practical to share every expense as it occurs. Thus individuals in the group pay for particular things, such as meals, hotels, taxi rides, and plane tickets. After the trip, each student's expenses are tallied and money is exchanged so that the net cost to each is the same, to within one cent. In the past, this money exchange has been tedious and time consuming. Your job is to compute, from a list of expenses, the minimum amount of money that must change hands in order to equalize (within one cent) all the students' costs.

Input

Standard input will contain the information for several trips. Each trip consists of a line containing a positive integer n denoting the number of students on the trip. This is followed by n lines of input, each containing the amount spent by a student in dollars and cents. There are no more than 1000 students and no student spent more than \$10,000.00. A single line containing 0 follows the information for the last trip.

Output

For each trip, output a line stating the total amount of money, in dollars and cents, that must be exchanged to equalize the students' costs.

Sample Input

```
3
10.00
20.00
30.00
4
15.00
15.01
3.00
3.01
0
```

Sample Output

```
$10.00
$11.99
```

1.6.4 LCD Display

PC/UVa IDs: 110104/706, **Popularity:** A, **Success rate:** average **Level:** 1

A friend of yours has just bought a new computer. Before this, the most powerful machine he ever used was a pocket calculator. He is a little disappointed because he liked the LCD display of his calculator more than the screen on his new computer! To make him happy, write a program that prints numbers in LCD display style.

Input

The input file contains several lines, one for each number to be displayed. Each line contains integers s and n, where n is the number to be displayed ($0 \leq n \leq 99,999,999$) and s is the size in which it shall be displayed ($1 \leq s \leq 10$). The input will be terminated by a line containing two zeros, which should not be processed.

Output

Print the numbers specified in the input file in an LCD display-style using s "-" signs for the horizontal segments and s "|" signs for the vertical ones. Each digit occupies exactly $s + 2$ columns and $2s + 3$ rows. Be sure to fill all the white space occupied by the digits with blanks, including the last digit. There must be exactly one column of blanks between two digits.

Output a blank line after each number. You will find an example of each digit in the sample output below.

Sample Input

```
2 12345
3 67890
0 0
```

Sample Output

```
    --   --         --
  |    |    | |  | |
  |    |    | |  | |
     --   --   --   --
  | |       |    |    |
  | |       |    |    |
     --   --         --

 ---   ---   ---   ---   ---
|         | |   | |   | |   |
|         | |   | |   | |   |
|         | |   | |   | |   |
 ---         ---   ---
|   |     | |   |     | |   |
|   |     | |   |     | |   |
|   |     | |   |     | |   |
 ---         ---   ---   ---
```

1.6.5 Graphical Editor

PC/UVa IDs: 110105/10267, **Popularity:** B, **Success rate:** low **Level:** 1

Graphical editors such as Photoshop allow us to alter bit-mapped images in the same way that text editors allow us to modify documents. Images are represented as an $M \times N$ array of pixels, where each pixel has a given color.

Your task is to write a program which simulates a simple interactive graphical editor.

Input

The input consists of a sequence of editor commands, one per line. Each command is represented by one capital letter placed as the first character of the line. If the command needs parameters, they will be given on the same line separated by spaces.

Pixel coordinates are represented by two integers, a column number between $1 \ldots M$ and a row number between $1 \ldots N$, where $1 \leq M, N \leq 250$. The origin sits in the upper-left corner of the table. Colors are specified by capital letters.

The editor accepts the following commands:

I M N	Create a new $M \times N$ image with all pixels initially colored white (`O`).
C	Clear the table by setting all pixels white (`O`). The size remains unchanged.
L X Y C	Colors the pixel (X, Y) in color (`C`).
V X Y1 Y2 C	Draw a vertical segment of color (`C`) in column X, between the rows $Y1$ and $Y2$ inclusive.
H X1 X2 Y C	Draw a horizontal segment of color (`C`) in the row Y, between the columns $X1$ and $X2$ inclusive.
K X1 Y1 X2 Y2 C	Draw a filled rectangle of color `C`, where $(X1, Y1)$ is the upper-left and $(X2, Y2)$ the lower right corner.
F X Y C	Fill the region R with the color `C`, where R is defined as follows. Pixel (X, Y) belongs to R. Any other pixel which is the same color as pixel (X, Y) and shares a common side with any pixel in R also belongs to this region.
S Name	Write the file name in MSDOS 8.3 format followed by the contents of the current image.
X	Terminate the session.

Output

On every command `S NAME`, print the filename $NAME$ and contents of the current image. Each row is represented by the color contents of each pixel. See the sample output.

Ignore the entire line of any command defined by a character other than `I, C, L, V, H, K, F, S,` or `X`, and pass on to the next command. In case of other errors, the program behavior is unpredictable.

Sample Input

```
I 5 6
L 2 3 A
S one.bmp
G 2 3 J
F 3 3 J
V 2 3 4 W
H 3 4 2 Z
S two.bmp
X
```

Sample Output

```
one.bmp
OOOOO
OOOOO
OAOOO
OOOOO
OOOOO
OOOOO
two.bmp
JJJJJ
JJZZJ
JWJJJ
JWJJJ
JJJJJ
JJJJJ
```

1.6.6 Interpreter

PC/UVa IDs: 110106/10033, **Popularity:** B, **Success rate:** low **Level:** 2

A certain computer has ten registers and 1,000 words of RAM. Each register or RAM location holds a three-digit integer between 0 and 999. Instructions are encoded as three-digit integers and stored in RAM. The encodings are as follows:

`100`	means *halt*
`2`*dn*	means *set register d to n (between 0 and 9)*
`3`*dn*	means *add n to register d*
`4`*dn*	means *multiply register d by n*
`5`*ds*	means *set register d to the value of register s*
`6`*ds*	means *add the value of register s to register d*
`7`*ds*	means *multiply register d by the value of register s*
`8`*da*	means *set register d to the value in RAM whose address is in register a*
`9`*sa*	means *set the value in RAM whose address is in register a to that of register s*
`0`*ds*	means *goto the location in register d unless register s contains 0*

All registers initially contain 000. The initial content of the RAM is read from standard input. The first instruction to be executed is at RAM address 0. All results are reduced modulo 1,000.

Input

The input begins with a single positive integer on a line by itself indicating the number of cases, each described as below. This is followed by a blank line, and there will be a blank line between each two consecutive inputs.

Each input case consists of up to 1,000 three-digit unsigned integers, representing the contents of consecutive RAM locations starting at 0. Unspecified RAM locations are initialized to 000.

Output

The output of each test case is a single integer: the number of instructions executed up to and including the *halt* instruction. You may assume that the program does halt. Separate the output of two consecutive cases by a blank line.

Sample Input

```
1

299
492
495
```

```
399
492
495
399
283
279
689
078
100
000
000
000
```

Sample Output

```
16
```

1.6.7 Check the Check

PC/UVa IDs: 110107/10196, **Popularity:** B, **Success rate:** average **Level:** 1

Your task is to write a program that reads a chessboard configuration and identifies whether a king is under attack (in check). A king is in check if it is on square which can be taken by the opponent on his next move.

White pieces will be represented by uppercase letters, and black pieces by lowercase letters. The white side will always be on the bottom of the board, with the black side always on the top.

For those unfamiliar with chess, here are the movements of each piece:

Pawn (`p` or `P`): can only move straight ahead, one square at a time. However, it takes pieces diagonally, and that is what concerns you in this problem.

Knight (`n` or `N`) : has an L-shaped movement shown below. It is the only piece that can jump over other pieces.

Bishop (`b` or `B`) : can move any number of squares diagonally, either forward or backward.

Rook (`r` or `R`) : can move any number of squares vertically or horizontally, either forward or backward.

Queen (`q` or `Q`) : can move any number of squares in any direction (diagonally, horizontally, or vertically) either forward or backward.

King (`k` or `K`) : can move one square at a time in any direction (diagonally, horizontally, or vertically) either forward or backward.

Movement examples are shown below, where "`*`" indicates the positions where the piece can capture another piece:

```
Pawn        Rook        Bishop      Queen       King        Knight
........    ...*....    .......*    ...*...*    ........    ........
........    ...*....    *.....*.    *..*..*.    ........    ........
........    ...*....    .*...*..    .*.*.*..    ........    ..*.*...
........    ...*....    ..*.*...    ..***...    ..***...    .*...*..
...p....    ***r****    ...b....    ***q****    ..*k*...    ...n....
..*.*...    ...*....    ..*.*...    ..***...    ..***...    .*...*..
........    ...*....    .*...*..    .*.*.*..    ........    ..*.*...
........    ...*....    *.....*.    *..*..*.    ........    ........
```

Remember that the knight is the only piece that can jump over other pieces. The pawn movement will depend on its side. If it is a black pawn, it can only move one square diagonally down the board. If it is a white pawn, it can only move one square diagonally up the board. The example above is a black pawn, described by a lowercase "`p`". We use "*move*" to indicate the squares where the pawn can capture another piece.

Input

There will be an arbitrary number of board configurations in the input, each consisting of eight lines of eight characters each. A "." denotes an empty square, while upper- and lowercase letters represent the pieces as defined above. There will be no invalid characters and no configurations where both kings are in check. You must read until you find an empty board consisting only of "." characters, which should not be processed. There will be an empty line between each pair of board configurations. All boards, except for the empty one, will contain exactly one white king and one black king.

Output

For each board configuration read you must output one of the following answers:
`Game #`d`: white king is in check.`
`Game #`d`: black king is in check.`
`Game #`d`: no king is in check.`
where d stands for the game number starting from 1.

Sample Input

```
..k.....
ppp.pppp
........
.R...B..
........
........
PPPPPPPP
K.......

rnbqk.nr
ppp..ppp
....p...
...p....
.bPP....
.....N..
PP..PPPP
RNBQKB.R

........
........
........
........
........
........
........
........
```

Sample Output

```
Game #1: black king is in check.
Game #2: white king is in check.
```

1.6.8 Australian Voting

PC/UVa IDs: 110108/10142, **Popularity:** B, **Success rate:** low **Level:** 1

Australian ballots require that voters rank all the candidates in order of choice. Initially only the first choices are counted, and if one candidate receives more than 50% of the vote then that candidate is elected. However, if no candidate receives more than 50%, all candidates tied for the lowest number of votes are eliminated. Ballots ranking these candidates first are recounted in favor of their highest-ranked non-eliminated candidate. This process of eliminating the weakest candidates and counting their ballots in favor of the preferred non-eliminated candidate continues until one candidate receives more than 50% of the vote, or until all remaining candidates are tied.

Input

The input begins with a single positive integer on a line by itself indicating the number of cases following, each as described below. This line is followed by a blank line. There is also a blank line between two consecutive inputs.

The first line of each case is an integer $n \leq 20$ indicating the number of candidates. The next n lines consist of the names of the candidates in order, each up to 80 characters in length and containing any printable characters. Up to 1,000 lines follow, each containing the contents of a ballot. Each ballot contains the numbers from 1 to n in some order. The first number indicates the candidate of first choice; the second number indicates candidate of second choice, and so on.

Output

The output of each test case consists of either a single line containing the name of the winner or several lines containing the names of all candidates who are tied. The output of each two consecutive cases are separated by a blank line.

Sample Input

```
1

3
John Doe
Jane Smith
Jane Austen
1 2 3
2 1 3
2 3 1
1 2 3
3 1 2
```

Sample Output

```
John Doe
```

1.7 Hints

1.3 Who should get the extra pennies if the total does not divide evenly?

1.5 How do we best handle the fill command? Is it easier to keep separate copies of the old and new image?

1.8 Notes

1.1 The $3n + 1$ (or Collatz) problem remains open to this day. See Lagarias [Lag85] for an excellent mathematical survey. An international conference on the Collatz problem was held in 1999; see *http://www.math.grinnell.edu/~chamberl/conf.html* for the online proceedings.

1.2 The Minesweeper consistency problem takes as input an $n \times n$ rectangular grid with the squares labeled by numbers 0 to 8, mines, or left blank, and asks: is there a configuration of mines in the grid that result in the given pattern of symbols, according to the usual Minesweeper rules? The Clay Institute of Mathematics (*http://www.claymath.org/*) offers a $1,000,000 prize for an efficient algorithm which solves this problem.

But don't get too excited! The Minesweeper consistency problem has been proven NP-complete [Kay00], meaning that no efficient algorithm for it can exist without revolutionizing our understanding of computation. See [GJ79] for a thorough discussion of NP-completeness.

1.6 Software-interpreted virtual machines are the key to the portability of languages such as Java. It is an interesting project to write a simulator for the machine language of an old, obsolete, but simple computer such as the PDP-8. Today's hardware is so much faster that your virtual PDP-8 will significantly outrun the original machine!

1.7 Once you have written a legal move generator (the essence of this problem) you are not very far from building your own chess-playing program! See [New96, Sch97] for stories of how computer chess and checkers programs work and beat the human World Champions at their own games.

1.8 The mathematics of voting systems is a fascinating subject. Arrow's *impossibility theorem* states that no voting system can simultaneously satisfy all of five obviously desirable properties. See [COM94] for an interesting discussion of the mathematics of social choice.

2
Data Structures

Data structures are the heart of any sophisticated program. Selecting the right data structure can make an enormous difference in the complexity of the resulting implementation. Pick the right data representation, and your task will be easy to program. Pick the wrong representation, and you may spend enormous amounts of time and code covering up for your lousy initial decision.

In this chapter, we will review the fundamental data structures which every programmer should be familiar with. We will motivate this discussion with an example program based on a children's card game. Many classic programming problems are based on games. After all, who makes it through their introductory programming courses without moving the towers of Hanoi, taking the knight's tour, or solving the eight queens problem?

2.1 Elementary Data Structures

Here we present the abstract operations of the most important data structures: stacks, queues, dictionaries, priority queues, and sets. We also describe the simplest way to implement these structures from scratch.

Be aware, however, that modern object-oriented languages like C++ and Java come with standard libraries of fundamental data structures. These will be briefly described in Section 2.2. It is worth every programmer's time to get acquainted with your libraries instead of repeatedly reinventing the wheel. Once you do, you can read this section with an eye for what each data structure is good for instead of the details of how it should be implemented.

2.1.1 Stacks

Stacks and queues are containers where items are retrieved according to the order of insertion, independent of content. *Stacks* maintain *last-in, first-out* order. The abstract operations on a stack include –

- *Push(x,s)* — Insert item x at the top of stack s.
- *Pop(s)* — Return (and remove) the top item of stack s.
- *Initialize(s)* — Create an empty stack.
- *Full(s), Empty(s)* — Test whether the stack can accept more pushes or pops, respectively.

Note that there is no element search operation defined on standard stacks and queues.

Defining these abstract operations enables us build a stack module to use and reuse without knowing the details of the implementation. The easiest implementation uses an array with an index variable to represent the top of the stack. An alternative implementation, using linked lists, is better because it can't overflow.

Stacks naturally model piles of objects, such as dinner plates. After a new plate is washed, it is pushed on the top of the stack. When someone is hungry, a clean plate is popped off the top. A stack is an appropriate data structure for this task since the plates don't care which one is used next. Thus one important application of stacks is whenever order *doesn't* matter, because stacks are particularly simple containers to implement.

Stack order is important in processing any properly nested structure. This includes parenthesized formulas (push on a "(", pop on ")"), recursive program calls (push on a procedure entry, pop on a procedure exit), and depth-first traversals of graphs (push on discovering a vertex, pop on leaving it for the last time).

2.1.2 Queues

Queues maintain *first-in, first-out* order. This appears fairer than last-in, first-out, which is why lines at stores are organized as queues instead of stacks. Decks of playing cards can be modeled by queues, since we deal the cards off the top of the deck and add them back in at the bottom. FIFO queues will be used in implementing breadth-first search in graphs in Chapter 9.

The abstract operations on a queue include —

- *Enqueue(x,q)* — Insert item x at the back of queue q.
- *Dequeue(q)* — Return (and remove) the front item from queue q
- *Initialize(q), Full(q), Empty(q)* — Analogous to these operation on stacks.

Queues are more difficult to implement than stacks, because action happens at both ends. The *simplest* implementation uses an array, inserting new elements at one end and *moving* all remaining elements to fill the empty space created on each dequeue.

However, it is very wasteful to move all the elements on each dequeue. How can we do better? We can maintain indices to the first (head) and last (tail) elements in the

array/queue and do all operations locally. There is no reason why we must explicitly clear previously used cells, although we leave a trail of garbage behind the previously dequeued items.

Circular queues let us reuse this empty space. Note that the pointer to the front of the list is always *behind* the back pointer! When the queue is full, the two indices will point to neighboring or identical elements. There are several possible ways to adjust the indices for circular queues. *All are tricky!* The easiest solution distinguishes full from empty by maintaining a count of how many elements exist in the queue:

```
typedef struct {
        int q[QUEUESIZE+1];             /* body of queue */
        int first;                      /* position of first element */
        int last;                       /* position of last element */
        int count;                      /* number of queue elements */
} queue;

init_queue(queue *q)
{
        q->first = 0;
        q->last = QUEUESIZE-1;
        q->count = 0;
}

enqueue(queue *q, int x)
{
        if (q->count >= QUEUESIZE)
                printf("Warning: queue overflow enqueue x=%d\n",x);
        else {
                q->last = (q->last+1) % QUEUESIZE;
                q->q[ q->last ] = x;
                q->count = q->count + 1;
        }
}

int dequeue(queue *q)
{
        int x;

        if (q->count <= 0) printf("Warning: empty queue dequeue.\n");
        else {
                x = q->q[ q->first ];
                q->first = (q->first+1) % QUEUESIZE;
                q->count = q->count - 1;
        }
```

```
        return(x);
}

int empty(queue *q)
{
        if (q->count <= 0) return (TRUE);
        else return (FALSE);
}
```

Queues are one of the few data structures which are easier to program using linked lists than arrays, because they eliminate the need to test for the wrap-around condition.

2.1.3 Dictionaries

Dictionaries permit content-based retrieval, unlike the position-based retrieval of stacks and queues. They support three primary operations –

- *Insert(x,d)* — Insert item x into dictionary d.
- *Delete(x,d)* — Remove item x (or the item pointed to by x) from dictionary d.
- *Search(k,d)* — Return an item with key k if one exists in dictionary d.

A data structures course may well present a dozen different ways to implement dictionaries, including sorted/unsorted linked lists, sorted/unsorted arrays, and a forest full of random, splay, AVL, and red-black trees – not to mention all the variations on hashing.

The primary issue in algorithm analysis is performance, namely, achieving the best possible trade-off between the costs of these three operations. But what we usually want in practice is the simplest way to get the job done under the given time constraints. The right answer depends on how much the contents of your dictionary change over the course of execution:

- *Static Dictionaries* — These structures get built once and never change. Thus they need to support search, but not insertion on deletion.

  The right answer for static dictionaries is typically an array. The only real question is whether to keep it sorted, in order to use binary search to provide fast membership queries. Unless you have tight time constraints, it probably isn't worth using binary search until $n > 100$ or so. You might even get away with sequential search to $n = 1,000$ or more, provided you will not be doing too many searches.

  Sorting algorithms and binary search always prove harder to debug than they should. Library sort/search routines are available for C, C++, and Java, and will be presented in Chapter 4.

- *Semi-dynamic Dictionaries* — These structures support insertion and search queries, but not deletion. If we know an upper bound on the number of elements to be inserted we can use an array, but otherwise we must use a linked structure.

Hash tables are excellent dictionary data structures, particularly if deletion need not be supported. The idea is to apply a function to the search key so we can determine *where* the item will appear in an array without looking at the other items. To make the table of reasonable size, we must allow for *collisions*, two distinct keys mapped to the same location.

The two components to hashing are (1) defining a hash function to map keys to integers in a certain range, and (2) setting up an array as big as this range, so that the hash function value can specify an index.

The basic hash function converts the key to an integer, and takes the value of this integer mod the size of the hash table. Selecting a table size to be a prime number (or at least avoiding obvious composite numbers like 1,000) is helpful to avoid trouble. Strings can be converted to integers by using the letters to form a base "alphabet-size" number system. To convert "steve" to a number, observe that e is the 5th letter of the alphabet, s is the 19th letter, t is the 20th letter, and v is the 22nd letter. Thus "steve" $\Rightarrow 26^4 \times 19 + 26^3 \times 20 + 26^2 \times 5 + 26^1 \times 22 + 26^0 \times 5 = 9,038,021$. The first, last, or middle ten characters or so will likely suffice for a good index. Tricks on how to do the modular arithmetic efficiently will be discussed in Chapter 7.

The absence of deletion makes *open addressing* a nice, easy way to resolve collisions. In open addressing, we use a simple rule to decide where to put a new item when the desired space is already occupied. Suppose we always put it in the next unoccupied cell. On searching for a given item, we go to the intended location and search sequentially. If we find an empty cell before we find the item, it does not exist anywhere in the table.

Deletion in an open addressing scheme is very ugly, since removing one element can break a chain of insertions, making some elements inaccessible. The key to efficiency is using a large-enough table that contains many holes. Don't be cheap in selecting your table size or else you will pay the price later.

- *Fully Dynamic Dictionaries* — Hash tables are great for fully dynamic dictionaries as well, provided we use *chaining* as the collision resolution mechanism. Here we associate a linked list with each table location, so insertion, deletion, and query reduce to the same problem in linked lists. If the hash function does a good job the m keys will be distributed uniformly in a table of size n, so each list will be short enough to search quickly.

2.1.4 Priority Queues

Priority queues are data structures on sets of items supporting three operations –

- *Insert(x,p)* — Insert item x into priority queue p.
- *Maximum(p)* — Return the item with the largest key in priority queue p.
- *ExtractMax(p)* — Return and remove the item with the largest key in p.

Priority queues are used to maintain schedules and calendars. They govern who goes next in simulations of airports, parking lots, and the like, whenever we need to schedule events according to a clock. In a human life simulation, it may be most convenient to determine when someone will die immediately after they are born. We can then stick this date in a priority queue so as to be reminded when to bury them.

Priority queues are used to schedule events in the sweep-line algorithms common to computational geometry. Typically, we use the priority queue to store the points we have not yet encountered, ordered by x-coordinate, and push the line forward one step at a time.

The most famous implementation of priority queues is the binary heap, which can be efficiently maintained in either a top-down or bottom-up manner. Heaps are very slick and efficient, but may be tricky to get right under time pressure. Far simpler is maintaining a sorted array, particularly if you will not be performing too many insertions.

2.1.5 Sets

Sets (or more strictly speaking *subsets*) are unordered collections of elements drawn from a given universal set U. Set data structures get distinguished from dictionaries because there is at least an implicit need to encode which elements from U are *not* in the given subset.

The basic operations on subsets are —

- *Member(x,S)* — Is an item x an element of subset S?
- *Union(A,B)* — Construct subset $A \cup B$ of all elements in subset A or in subset B.
- *Intersection(A,B)* — Construct subset $A \cap B$ of all elements in subset A and in subset B.
- *Insert(x,S), Delete(x,S)* — Insert/delete element x into/from subset S.

For sets of a large or unbounded universe, the obvious solution is representing a subset using a dictionary. Using sorted dictionaries makes union and intersection operations much easier, basically reducing the problem to merging two sorted dictionaries. An element is in the union if it appears at least once in the merged list, and in the intersection if it appears exactly twice.

For sets drawn from small, unchanging universes, bit vectors provide a convenient representation. An n-bit vector or array can represent any subset S from an n-element universe. Bit i will be 1 iff $i \in S$. Element insertion and deletion operations simply flip the appropriate bit. Intersection and union are done by "and-ing" or "or-ing" the corresponding bits together. Since only one bit is used per element, bit vectors can be space efficient for surprisingly large values of $|U|$. For example, an array of 1,000 standard four-byte integers can represent any subset on 32,000 elements.

2.2 Object Libraries

Users of modern object-oriented languages such as C++ and Java have implementations of these basic data structures available using standard library classes.

2.2.1 The C++ Standard Template Library

A C library of general-purpose data structures, like stacks and queues, cannot really exist because functions in C can't tell the type of their arguments. Thus we would have to define separate routines such as `push_int()` and `push_char()` for every possible data type. Further, such an approach couldn't generalize to construct stacks on user-defined data types.

Templates are C++'s mechanism to define abstract objects which can be parameterized by type. The C++ *Standard Template Library (STL)* provides implementations of all the data structures defined above and much more. Each data object must have the type of its elements fixed (i.e., templated) at compilation time, so

```
#include <stl.h>

  stack<int> S;
  stack<char> T;
```

declares two stacks with different element types.

Good references on STL include [MDS01] and *http://www.sgi.com/tech/stl/*. Brief descriptions of our featured data structures follow below –

- *Stack* — Methods include `S.push()`, `S.top()`, `S.pop()`, and `S.empty()`. Top returns but does not remove the element on top; while pop removes but does not return the element. Thus always top on pop [Seu63]. Linked implementations ensure the stack will never be full.
- *Queue* — Methods include `Q.front()`, `Q.back()`, `Q.push()`, `Q.pop()`, and `Q.empty()` and have the same idiosyncrasies as stack.
- *Dictionaries* — STL contains a wide variety of containers, including `hash_map`, a hashed associative container binding keys to data items. Methods include `H.erase()`, `H.find()`, and `H.insert()`.
- *Priority Queues* — Declared `priority_queue<int> Q;`, methods include `Q.top()`, `Q.push()`, `Q.pop()`, and `Q.empty()`.
- *Sets* — Sets are represented as sorted associative containers, declared `set<key, comparison> S;`. Set algorithms include `set_union` and `set_intersection`, as well as other standard set operators.

2.2.2 The Java `java.util` *Package*

Useful standard Java objects appear in the `java.util` package. Almost all of `java.util` is available on the judge, except for a few libraries which provide

an unhealthy degree of power to the contestant. For details, see Sun's website *http://java.sun.com/products/jdk*.

The collection of all Java classes defines an inheritance hierarchy, which means that subclasses are built on superclasses by adding methods and variables. As you move up the inheritance hierarchy, classes become more general and more abstract. The sole purpose of an *abstract* class is to provide an appropriate superclass from which other classes may inherit its interface and/or implementation. The abstract classes can only declare objects, not instantiate them. Classes from which objects can be instantiated are called *concrete* classes.

If you want to declare a general data structure object, declare it with an interface or abstract class and instantiate it using a concrete class. For example,

```
Map myMap = new HashMap();
```

In this case, `myMap` is treated as an object of `Map` class. Otherwise, you can just declare and instantiate an object with concrete class, like

```
HashMap myMap = new HashMap();
```

Here, `myMap` is only an object of `HashMap` class.

To use `java.util` include `import java.util.*;` at the beginning of your program to import the whole package, or replace the star to import a specific class, like `import java.util.HashMap;`.

Appropriate implementations of the basic data structures include —

Data Structure	Abstract class	Concrete class	Methods
Stack	No interface	`Stack`	`pop`, `push`, `empty`, `peek`
Queue	`List`	`ArrayList`, `LinkedList`	`add`, `remove`, `clear`
Dictionaries	`Map`	`HashMap`, `Hashtable`	`put`, `get`, `contains`
Priority Queue	`SortedMap`	`TreeMap`	`firstKey`, `lastKey`, `headMap`
Sets	`Set`	`HashSet`	`add`, `remove`, `contains`

2.3 Program Design Example: Going to War

In the children's card game War, a standard 52-card deck is dealt to two players (1 and 2) such that each player has 26 cards. Players do not look at their cards, but keep them in a packet face down. The object of the game is to win all the cards.

Both players play by turning their top cards face up and putting them on the table. Whoever turned the higher card takes both cards and adds them (face down) to the bottom of their packet. Cards rank as usual from high to low: *A, K, Q, J, T, 9, 8, 7, 6, 5, 4, 3, 2*. Suits are ignored. Both players then turn up their next card and repeat. The game continues until one player wins by taking all the cards.

When the face up cards are of equal rank there is a war. These cards stay on the table as both players play the next card of their pile face down and then another card face up. Whoever has the higher of the new face up cards wins the war, and adds all six cards to the bottom of his or her packet. If the new face up cards are equal as well, the war continues: each player puts another card face down and one face up. The war

goes on like this as long as the face up cards continue to be equal. As soon as they are different, the player with the higher face up card wins all the cards on the table.

If someone runs out of cards in the middle of a war, the other player automatically wins. The cards are added to the back of the winner's hand in the exact order they were dealt, specifically 1's first card, 2's first card, 1's second card, etc.

As anyone with a five year-old nephew knows, a game of War can take a long time to finish. But how long? Your job is to write a program to simulate the game and report the number of moves.

Solution starts below

How do we read such a problem description? Keep the following in mind as you design, code, test, and debug your solutions:

- *Read the Problem Carefully* — Read each line of the problem statement carefully, and reread it when the judge complains about an error. Skim the passage first, since much of the description may be background/history that does not impact the solution. Pay particular attention to the input and output descriptions, and sample input/output, but ...
- *Don't Assume* — Reading and understanding specifications is an important part of contest (and real-life) programming. Specifications often leave unspecified traps to fall in.

  Just because certain examples exhibit some nice property does not mean that all the test data will. Be on the lookout for unspecified input orders, unbounded input numbers, long line lengths, negative numbers, etc. Any input which is not explicitly forbidden should be assumed to be permitted!
- *Not So Fast, Louie* — Efficiency is often not an important issue, unless we are using exponential algorithms for problems where polynomial algorithms suffice. Don't worry too much about efficiency unless you have seen or can predict trouble. Read the specification to learn the maximum possible input size, and decide whether the most straightforward algorithm will suffice on such inputs.

  Even though a game of war seems interminable when you are playing with your nephew (and in fact can go on forever), we see no reason to worry about efficiency with this particular problem description.

2.4 Hitting the Deck

What is the best data structure to represent a deck of cards? The answer depends upon what you are going to do with them. Are you going to shuffle them? Compare their values? Search for patterns in the deck? Your intended actions define the operations of the data structure.

The primary action we need from our deck is dealing cards out from the top and adding them to the rear of our deck. Thus it is natural to represent each player's hand using the FIFO queue we defined earlier.

But there is an even more fundamental problem. How do we represent each card? Remember that cards have both suits (clubs, diamonds, hearts, and spades) and values (ace, 2–10, jack, queen, king). We have several possible choices. We can represent each card by a pair of characters or numbers specifying the suit and value. In the war problem, we might even throw away the suit entirely – but such thinking can get us in trouble. What if we had to print the winning card, or needed strong evidence that our queue implementation was working perfectly? An alternate approach might represent each card with a distinct integer, say 0 to 51, and map in back and forth between numbers and cards as needed.

The primary operation in war is comparing cards by their face value. This is tricky to do with the first character representation, because we must compare according to the historical but arbitrary ordering of face cards. Ad hoc logic might seem necessary to deal with this problem.

Instead, we will present the mapping approach as a generally useful programming technique. Whenever we can create a numerical *ranking* function and a dual *unranking* function which hold over a particular set of items $s \in S$, we can represent any item by its integer rank. The key property is that $s = unrank(rank(s))$. Thus the ranking function can be thought of as a hash function without collisions.

How can we rank and unrank playing cards? We order the card values from lowest to highest, and note that there are four distinct cards of each value. Multiplication and division are the key to mapping them from 0 to 51:

```
#define NCARDS  52      /* number of cards */
#define NSUITS  4       /* number of suits */

char values[] = "23456789TJQKA";
char suits[] = "cdhs";

int rank_card(char value, char suit)
{
        int i,j;        /* counters */

        for (i=0; i<(NCARDS/NSUITS); i++)
                if (values[i]==value)
                        for (j=0; j<NSUITS; j++)
                                if (suits[j]==suit)
                                        return( i*NSUITS + j );

        printf("Warning: bad input value=%d, suit=%d\n",value,suit);
}

char suit(int card)
{
        return( suits[card % NSUITS] );
}
```

```
char value(int card)
{
        return( values[card/NSUITS] );
}
```

Ranking and unranking functions are easy to develop for permutations, subsets, and most combinatorial objects. They are a general programming technique which can simplify operations on many different types of data.

2.5 String Input/Output

For our war program, the input consists of pairs of lines for each input deck, the first corresponding to player 1's cards, the second to player 2's cards. Here is an example of three games' worth of cards:

```
4d Ks As 4h Jh 6h Jd Qs Qh 6s 6c 2c Kc 4s Ah 3h Qd 2h 7s 9s 3c 8h Kd 7h Th Td
8d 8c 9c 7c 5d 4c Js Qc 5s Ts Jc Ad 7d Kh Tc 3s 8s 2d 2s 5h 6d Ac 5c 9h 3d 9d
6d 9d 8c 4s Kc 7c 4d Tc Kd 3s 5h 2h Ks 5c 2s Qh 8d 7d 3d Ah Js Jd 4c Jh 6c Qc
9h Qd Qs 9s Ac 8h Td Jc 7s 2d 6s As 4h Ts 6h 2c Kh Th 7h 5s 9c 5d Ad 3h 8s 3c
Ah As 4c 3s 7d Jc 5h 8s Qc Kh Td 3h 5c 9h 8c Qs 3d Ks 4d Kd 6c 6s 7h Qh 3c Jd
2h 8h 7s 2c 5d 7c 2d Tc Jh Ac 9s 9c 5s Qd 4s Js 6d Kc 2s Th 8d 9d 4h Ad 6h Ts
```

Many problems require reading non-numerical data. Text strings will be better discussed in Chapter 3, but note that you have several choices when it comes to reading text input:

- You can repeatedly get single characters from the input stream (e.g., `getchar()` in C), and process them one at a time.
- You can repeatedly get formatted tokens (e.g., `scanf()` in C) and process them as needed.
- You can read the entire line as a single string (e.g., `gets()` in C), and then parse it by accessing characters/substring in the string.
- You can use modern input/output primitives such as streams if your language supports them. Of course, you are still faced with the problem of deciding whether you want characters, strings, or something else as the basic input block.

In our war implementation, we chose the first option – namely, reading characters sequentially and processing them one by one. To make the example more illustrative, we explicitly test for end of line (`'\n'` in C):

```
main()
{
        queue decks[2];                 /* player's decks */
        char value,suit,c;              /* input characters */
        int i;                          /* deck counter */
```

```
        while (TRUE) {
            for (i=0; i<=1; i++) {
                init_queue(&decks[i]);

                while ((c = getchar()) != '\n') {
                    if (c == EOF) return;
                    if (c != ' ') {
                        value = c;
                        suit = getchar();
                        enqueue(&decks[i],rank_card(value,suit));
                    }
                }
            }

            war(&decks[0],&decks[1]);
        }
}
```

Note that we represented the two decks as an array of queues instead of as two separate queue variables. This way, we eliminate the need to duplicate all the input processing code for each deck.

2.6 Winning the War

With the proper scaffolding afforded by our data structure design, the main routine becomes fairly straightforward. Note that the order in which the conquered cards get returned to the winning deck also can be modeled as a queue, so we get to use our abstract type once again:

```
war(queue *a, queue *b)
{
        int steps=0;                    /* step counter */
        int x,y;                        /* top cards */
        queue c;                        /* cards involved in the war */
        bool inwar;                     /* are we involved in a war? */

        inwar = FALSE;
        init_queue(&c);

        while ((!empty(a)) && (!empty(b) && (steps < MAXSTEPS))) {
                steps = steps + 1;
                x = dequeue(a);
                y = dequeue(b);
                enqueue(&c,x);
```

```
                enqueue(&c,y);
                if (inwar) {
                        inwar = FALSE;
                } else {
                        if (value(x) > value(y))
                                clear_queue(&c,a);
                        else if  (value(x) < value(y))
                                clear_queue(&c,b);
                        else if (value(y) == value(x))
                                inwar = TRUE;
                }
        }

        if (!empty(a) && empty(b))
                printf("a wins in %d steps \n",steps);
        else if (empty(a) && !empty(b))
                printf("b wins in %d steps \n",steps);
        else if (!empty(a) && !empty(b))
                printf("game tied after %d steps, |a|=%d |b|=%d \n",
                        steps,a->count,b->count);
        else
                printf("a and b tie in %d steps \n",steps);
}

clear_queue(queue *a, queue *b)
{
        while (!empty(a)) enqueue(b,dequeue(a));
}
```

2.7 Testing and Debugging

Debugging can be particularly frustrating with the robot judge, since you never get to see the test case on which your program failed. Thus you can't fake your way around a problem – you have to do it right.

This makes it very important to test your program systematically before submission. Catching stupid errors yourself will save you time in the long run, particularly in contest situations where incorrect submissions are penalized. Several ideas go into designing good test files:

- *Test the Given Input* — Most problem specifications include sample input and output. Often (but not always) they match each other. Getting the sample input right is a necessary but not sufficient condition for correctness.
- *Test Incorrect Input* — If the problem specification tells you that your program must take action on illegal inputs, be sure to test for such problematic instances.

- *Test Boundary Conditions* — Many bugs in programs are due to "off-by-one" errors. Explicitly test your code for such conditions as empty input, one item, two items, and values which are zero.
- *Test Instances Where You Know the Correct Answer* — A critical part of developing a good test case is being sure you know what the right answer is. Your test cases should focus on simple enough instances that you can solve them by hand. It is easy to be fooled into accepting plausible-looking output without completely analyzing the desired behavior.
- *Test Big Examples Where You Don't Know the Correct Answer* — Usually only small examples are solvable by hand. This makes it difficult to validate your program on larger inputs. Try a few easily constructed large instances, such as random data or the numbers 1 to n inclusive, just to make certain the program doesn't crash or do anything stupid.

Testing is the art of revealing bugs. Debugging is the art of exterminating them. We designed this programming problem and coded it up ourselves for the purpose of this example. Yet getting it working without bugs took us significantly longer than expected. This is no surprise, for all programmers are inherent optimists. But how can you avoid falling into such traps?

- *Get To Know Your Debugger* — Any reasonable programming environment comes with a source-level debugger, which lets you stop execution at given positions or logical conditions, look at the contents of a variable, and change its value to see what happens. Source-level debuggers are well worth their weight in print statements; learn to use them. The sooner you start, the more time and frustration you will save.
- *Display Your Data Structures* — At one point in debugging our War program, we had an off-by-one error in our priority queue. This could only be revealed by displaying the contents of the priority queue to see what was missing. Write special-purpose display routines for all non-trivial data structures, as debuggers often have a hard time making sense out of them.
- *Test Invariants Rigorously* — The card ranking and unranking functions are a potential source of error whenever they are not inverses of each other. An *invariant* is a property of the program which is true regardless of the input. A simple invariant test like

```
for (i=0; i<NCARDS; i++)
   if (i != rank_card(value(i), suit(i)))
        printf("Error: rank_card(%c,%c)=%d not %d\n", value(i),
                suit(i), rank_card(value(i), suit(i)), i);
```

  completely tests the correctness of the ranking and unranking functions.
- *Inspect Your Code* — Perhaps the single most powerful way to debug your program is to read it carefully. Bugs tend to infest code that is too ugly to read or too clever to understand.

- *Make Your Print Statements Mean Something* — Adding print statements to peer into the internals of your malfunctioning program is a necessary evil which can be minimized by effective use of a source-level debugger. But if you are going to add debugging print statements, make them as useful as possible. Be sure to display with the names of the variable and its position in the program as well as its value. It is easy to get lost in an enormous volume of debugging output, but the relevant lines can be quickly found by searching or greping through the output file.

  Comment out your debugging statements when you have fixed that part of the program, but don't be so fast to delete them. If you are like us, you will probably need them again.

- *Make Your Arrays a Little Larger Than Necessary* — Off-by-one errors are a particularly subtle and tricky class of bugs. Clear thinking and discipline are the right way avoid them. But thinking is hard and memory is cheap. We find it a useful crutch to define arrays one or two elements larger than *should* be necessary so as to minimize the consequences of any off-by-one error.

- *Make Sure Your Bugs are Really Bugs* — We spent some time tracking down an infinite loop in the War program, before discovering that random initial card orderings have an astonishingly high probability of creating a cycling condition where players forever alternate taking cards from each other. The culprit was the deterministic card order specified to collect the spoils of war, i.e., always interleaving player 1 and player 2's cards. In fact, there was no bug in the program at all!

  Please learn from this example. Always shuffle the pickup cards randomly to avoid having to play War with your nephew forever.

2.8 Problems

2.8.1 Jolly Jumpers

PC/UVa IDs: 110201/10038, **Popularity:** A, **Success rate:** average **Level:** 1

A sequence of $n > 0$ integers is called a *jolly jumper* if the absolute values of the differences between successive elements take on all possible values 1 through $n - 1$. For instance,

```
1 4 2 3
```

is a jolly jumper, because the absolute differences are 3, 2, and 1, respectively. The definition implies that any sequence of a single integer is a jolly jumper. Write a program to determine whether each of a number of sequences is a jolly jumper.

Input

Each line of input contains an integer $n < 3,000$ followed by n integers representing the sequence.

Output

For each line of input generate a line of output saying "`Jolly`" or "`Not jolly`".

Sample Input

```
4 1 4 2 3
5 1 4 2 -1 6
```

Sample Output

```
Jolly
Not jolly
```

2.8.2 Poker Hands

PC/UVa IDs: 110202/10315, **Popularity:** C, **Success rate:** average **Level:** 2

A poker deck contains 52 cards. Each card has a suit of either clubs, diamonds, hearts, or spades (denoted `C`, `D`, `H`, `S` in the input data). Each card also has a value of either 2 through 10, jack, queen, king, or ace (denoted `2`, `3`, `4`, `5`, `6`, `7`, `8`, `9`, `T`, `J`, `Q`, `K`, `A`). For scoring purposes card values are ordered as above, with 2 having the lowest and ace the highest value. The suit has no impact on value.

A poker hand consists of five cards dealt from the deck. Poker hands are ranked by the following partial order from lowest to highest.

High Card. Hands which do not fit any higher category are ranked by the value of their highest card. If the highest cards have the same value, the hands are ranked by the next highest, and so on.

Pair. Two of the five cards in the hand have the same value. Hands which both contain a pair are ranked by the value of the cards forming the pair. If these values are the same, the hands are ranked by the values of the cards not forming the pair, in decreasing order.

Two Pairs. The hand contains two different pairs. Hands which both contain two pairs are ranked by the value of their highest pair. Hands with the same highest pair are ranked by the value of their other pair. If these values are the same the hands are ranked by the value of the remaining card.

Three of a Kind. Three of the cards in the hand have the same value. Hands which both contain three of a kind are ranked by the value of the three cards.

Straight. Hand contains five cards with consecutive values. Hands which both contain a straight are ranked by their highest card.

Flush. Hand contains five cards of the same suit. Hands which are both flushes are ranked using the rules for High Card.

Full House. Three cards of the same value, with the remaining two cards forming a pair. Ranked by the value of the three cards.

Four of a Kind. Four cards with the same value. Ranked by the value of the four cards.

Straight Flush. Five cards of the same suit with consecutive values. Ranked by the highest card in the hand.

Your job is to compare several pairs of poker hands and to indicate which, if either, has a higher rank.

Input

The input file contains several lines, each containing the designation of ten cards: the first five cards are the hand for the player named "*Black*" and the next five cards are the hand for the player named "*White*".

Output

For each line of input, print a line containing one of the following:

```
Black wins.
White wins.
Tie.
```

Sample Input

```
2H 3D 5S 9C KD 2C 3H 4S 8C AH
2H 4S 4C 2D 4H 2S 8S AS QS 3S
2H 3D 5S 9C KD 2C 3H 4S 8C KH
2H 3D 5S 9C KD 2D 3H 5C 9S KH
```

Sample Output

```
White wins.
Black wins.
Black wins.
Tie.
```

2.8.3 Hartals

PC/UVa IDs: 110203/10050, **Popularity:** B, **Success rate:** high **Level:** 2

Political parties in Bangladesh show their muscle by calling for regular *hartals* (strikes), which cause considerable economic damage. For our purposes, each party may be characterized by a positive integer h called the *hartal parameter* that denotes the average number of days between two successive strikes called by the given party.

Consider three political parties. Assume $h_1 = 3$, $h_2 = 4$, and $h_3 = 8$, where h_i is the hartal parameter for party i. We can simulate the behavior of these three parties for $N = 14$ days. We always start the simulation on a Sunday. There are no hartals on either Fridays or Saturdays.

	1	2	3	4	5	6	7	8	9	10	11	12	13	14
Days	Su	Mo	Tu	We	Th	Fr	Sa	Su	Mo	Tu	We	Th	Fr	Sa
Party 1			x			x			x			x		
Party 2				x				x				x		
Party 3								x						
Hartals			1	2				3	4			5		

There will be exactly five hartals (on days 3, 4, 8, 9, and 12) over the 14 days. There is no hartal on day 6 since it falls on Friday. Hence we lose five working days in two weeks.

Given the hartal parameters for several political parties and the value of N, determine the number of working days lost in those N days.

Input

The first line of the input consists of a single integer T giving the number of test cases to follow. The first line of each test case contains an integer N ($7 \leq N \leq 3,650$), giving the number of days over which the simulation must be run. The next line contains another integer P ($1 \leq P \leq 100$) representing the number of political parties. The ith of the next P lines contains a positive integer h_i (which will never be a multiple of 7) giving the *hartal parameter* for party i ($1 \leq i \leq P$).

Output

For each test case, output the number of working days lost on a separate line.

Sample Input

```
2
14
3
3
4
```

```
8
100
4
12
15
25
40
```

Sample Output

```
5
15
```

2.8.4 Crypt Kicker

PC/UVa IDs: 110204/843, **Popularity:** B, **Success rate:** low **Level:** 2

A common but insecure method of encrypting text is to permute the letters of the alphabet. In other words, each letter of the alphabet is consistently replaced in the text by some other letter. To ensure that the encryption is reversible, no two letters are replaced by the same letter.

Your task is to decrypt several encoded lines of text, assuming that each line uses a different set of replacements, and that all words in the decrypted text are from a dictionary of known words.

Input

The input consists of a line containing an integer n, followed by n lowercase words, one per line, in alphabetical order. These n words compose the dictionary of words which may appear in the decrypted text. Following the dictionary are several lines of input. Each line is encrypted as described above.

There are no more than 1,000 words in the dictionary. No word exceeds 16 letters. The encrypted lines contain only lower case letters and spaces and do not exceed 80 characters in length.

Output

Decrypt each line and print it to standard output. If there are multiple solutions, any one will do. If there is no solution, replace every letter of the alphabet by an asterisk.

Sample Input

```
6
and
dick
jane
puff
spot
yertle
bjvg xsb hxsn xsb qymm xsb rqat xsb pnetfn
xxxx yyy zzzz www yyyy aaa bbbb ccc dddddd
```

Sample Output

```
dick and jane and puff and spot and yertle
**** *** **** *** **** *** **** *** ******
```

2.8.5 Stack 'em Up

PC/UVa IDs: 110205/10205, **Popularity:** B, **Success rate:** average **Level:** 1

The Big City has many casinos. In one of them, the dealer cheats. She has perfected several shuffles; each shuffle rearranges the cards in exactly the same way whenever it is used. A simple example is the "bottom card" shuffle, which removes the bottom card and places it at the top. By using various combinations of these known shuffles, the crooked dealer can arrange to stack the cards in just about any particular order.

You have been retained by the security manager to track this dealer. You are given a list of all the shuffles performed by the dealer, along with visual cues that allow you to determine which shuffle she uses at any particular time. Your job is to predict the order of the cards after a sequence of shuffles.

A standard playing card deck contains 52 cards, with 13 values in each of four suits. The values are named *2, 3, 4, 5, 6, 7, 8, 9, 10, Jack, Queen, King, Ace*. The suits are named *Clubs, Diamonds, Hearts, Spades*. A particular card in the deck can be uniquely identified by its value and suit, typically denoted $< value >$ of $< suit >$. For example, "9 of Hearts" or "King of Spades." Traditionally a new deck is ordered first alphabetically by suit, then by value in the order given above.

Input

The input begins with a single positive integer on a line by itself indicating the number of test cases, followed by a blank line. There is also a blank line between two consecutive inputs.

Each case consists of an integer $n \leq 100$, the number of shuffles that the dealer knows. Then follow n sets of 52 integers, each comprising all the integers from 1 to 52 in some order. Within each set of 52 integers, i in position j means that the shuffle moves the ith card in the deck to position j.

Several lines follow, each containing an integer k between 1 and n. These indicate that you have observed the dealer applying the kth shuffle given in the input.

Output

For each test case, assume the dealer starts with a new deck ordered as described above. After all the shuffles had been performed, give the names of the cards in the deck, in the new order. The output of two consecutive cases will be separated by a blank line.

Sample Input

```
1

2
2 1 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26
27 28 29 30 31 32 33 34 35 36 37 38 39 40 41 42 43 44 45 46 47 48 49 50 52 51
52 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26
```

```
27 28 29 30 31 32 33 34 35 36 37 38 39 40 41 42 43 44 45 46 47 48 49 50 51 1
1
2
```

Sample Output

```
King of Spades
2 of Clubs
4 of Clubs
5 of Clubs
6 of Clubs
7 of Clubs
8 of Clubs
9 of Clubs
10 of Clubs
Jack of Clubs
Queen of Clubs
King of Clubs
Ace of Clubs
2 of Diamonds
3 of Diamonds
4 of Diamonds
5 of Diamonds
6 of Diamonds
7 of Diamonds
8 of Diamonds
9 of Diamonds
10 of Diamonds
Jack of Diamonds
Queen of Diamonds
King of Diamonds
Ace of Diamonds
2 of Hearts
3 of Hearts
4 of Hearts
5 of Hearts
6 of Hearts
7 of Hearts
8 of Hearts
9 of Hearts
10 of Hearts
Jack of Hearts
Queen of Hearts
King of Hearts
Ace of Hearts
2 of Spades
3 of Spades
4 of Spades
5 of Spades
6 of Spades
7 of Spades
8 of Spades
9 of Spades
10 of Spades
Jack of Spades
Queen of Spades
Ace of Spades
3 of Clubs
```

2.8.6 Erdös Numbers

PC/UVa IDs: 110206/10044, **Popularity:** B, **Success rate:** low **Level:** 2

The Hungarian Paul Erdös (1913–1996) was one of the most famous mathematicians of the 20th century. Every mathematician having the honor of being a co-author of Erdös is well respected.

Unfortunately, not everybody got the chance to write a paper with Erdös, so the best they could do was publish a paper with somebody who had published a scientific paper with Erdös. This gave rise to the so-called *Erdös numbers*. An author who has jointly published with Erdös had Erdös number 1. An author who had not published with Erdös but with somebody with Erdös number 1 obtained Erdös number 2, and so on.

Your task is to write a program which computes Erdös numbers for a given set of papers and scientists.

Input

The first line of the input contains the number of scenarios. Each scenario consists of a paper database and a list of names. It begins with the line P N, where P and N are natural numbers. Following this line is the paper database, with P lines each containing the description of one paper specified in the following way:

```
Smith, M.N., Martin, G., Erdos, P.: Newtonian forms of prime factors
```

Note that umlauts, like "ö," are simply written as "o". After the P papers follow N lines with names. Such a name line has the following format:

```
Martin, G.
```

Output

For every scenario you are to print a line containing a string "`Scenario` i" (where i is the number of the scenario) and the author names together with their Erdös number of all authors in the list of names. The authors should appear in the same order as they appear in the list of names. The Erdös number is based on the papers in the paper database of this scenario. Authors which do not have any relation to Erdös via the papers in the database have Erdös number "`infinity`."

Sample Input

```
1
4 3
Smith, M.N., Martin, G., Erdos, P.: Newtonian forms of prime factors
Erdos, P., Reisig, W.: Stuttering in petri nets
Smith, M.N., Chen, X.: First order derivates in structured programming
Jablonski, T., Hsueh, Z.: Selfstabilizing data structures
```

```
Smith, M.N.
Hsueh, Z.
Chen, X.
```

Sample Output

```
Scenario 1
Smith, M.N. 1
Hsueh, Z. infinity
Chen, X. 2
```

2.8.7 Contest Scoreboard

PC/UVa IDs: 110207/10258, **Popularity:** B, **Success rate:** average **Level:** 1

Want to compete in the ACM ICPC? Then you had better know how to keep score! Contestants are ranked first by the number of problems solved (the more the better), then by decreasing amounts of penalty time. If two or more contestants are tied in both problems solved and penalty time, they are displayed in order of increasing team numbers.

A problem is considered solved by a contestant if any of the submissions for that problem was judged correct. Penalty time is computed as the number of minutes it took until the first correct submission for a problem was received, plus 20 minutes for each incorrect submission prior to the correct solution. Unsolved problems incur no time penalties.

Input

The input begins with a single positive integer on a line by itself indicating the number of cases, each described as below. This line is followed by a blank line. There is also a blank line between two consecutive inputs.

The input consists of a snapshot of the judging queue, containing entries from some or all of contestants 1 through 100 solving problems 1 through 9. Each line of input consists of three numbers and a letter in the format *contestant problem time L*, where *L* can be `C`, `I`, `R`, `U`, or `E`. These stand for `C`orrect, `I`ncorrect, clarification `R`equest, `U`njudged, and `E`rroneous submission. The last three cases do not affect scoring.

The lines of input appear in the order in which the submissions were received.

Output

The output for each test case will consist of a scoreboard, sorted by the criteria described above. Each line of output will contain a contestant number, the number of problems solved by the contestant and the total time penalty accumulated by the contestant. Since not all contestants are actually participating, only display those contestants who have made a submission.

The output of two consecutive cases will be separated by a blank line.

Sample Input

```
1

1 2 10 I
3 1 11 C
1 2 19 R
1 2 21 C
1 1 25 C
```

Sample Output

```
1 2 66
3 1 11
```

2.8.8 Yahtzee

PC/UVa IDs: 110208/10149, **Popularity:** C, **Success rate:** average **Level:** 3

The game of Yahtzee involves five dice, which are thrown in 13 rounds. A score card contains 13 categories. Each round may be scored in a category of the player's choosing, but each category may be scored only once in the game. The 13 categories are scored as follows:

- **ones** - sum of all ones thrown
- **twos** - sum of all twos thrown
- **threes** - sum of all threes thrown
- **fours** - sum of all fours thrown
- **fives** - sum of all fives thrown
- **sixes** - sum of all sixes thrown
- **chance** - sum of all dice
- **three of a kind** - sum of all dice, provided at least three have same value
- **four of a kind** - sum of all dice, provided at least four have same value
- **five of a kind** - 50 points, provided all five dice have same value
- **short straight** - 25 points, provided four of the dice form a sequence (that is, 1,2,3,4 or 2,3,4,5 or 3,4,5,6)
- **long straight** - 35 points, provided all dice form a sequence (1,2,3,4,5 or 2,3,4,5,6)
- **full house** - 40 points, provided three of the dice are equal and the other two dice are also equal. (for example, 2,2,5,5,5)

Each of the last six categories may be scored as 0 if the criteria are not met.

The score for the game is the sum of all 13 categories plus a bonus of 35 points if the sum of the first six categories is 63 or greater.

Your job is to compute the best possible score for a sequence of rounds.

Input

Each line of input contains five integers between 1 and 6, indicating the values of the five dice thrown in each round. There are 13 such lines for each game, and there may be any number of games in the input data.

Output

Your output should consist of a single line for each game containing 15 numbers: the score in each category (in the order given), the bonus score (0 or 35), and the total score. If there is more than categorization that yields the same total score, any one will do.

Sample Input

```
1 2 3 4 5
1 2 3 4 5
1 2 3 4 5
1 2 3 4 5
1 2 3 4 5
1 2 3 4 5
1 2 3 4 5
1 2 3 4 5
1 2 3 4 5
1 2 3 4 5
1 2 3 4 5
1 2 3 4 5
1 2 3 4 5
1 1 1 1 1
6 6 6 6 6
6 6 6 1 1
1 1 1 2 2
1 1 1 2 3
1 2 3 4 5
1 2 3 4 6
6 1 2 6 6
1 4 5 5 5
5 5 5 5 6
4 4 4 5 6
3 1 3 6 3
2 2 2 4 6
```

Sample Output

```
1 2 3 4 5 0 15 0 0 0 25 35 0 0 90
3 6 9 12 15 30 21 20 26 50 25 35 40 35 327
```

2.9 Hints

2.2 Can we reduce the value of a poker hand to a single numerical value to make the comparison easier?

2.3 Do we need to build the actual table in order to compute the number of hartals?

2.4 Does it pay to partition the words into equivalence classes based on repeated letters and length?

2.7 What is the easiest way to sort on the multiple criteria?

2.8 Do we need to try all possible mappings of rounds to categories, or can we make certain assignments in a more straightforward way?

2.10 Notes

2.1 A jolly number is a special case of a *graceful graph* labeling. A graph is called graceful if there exists a way to label the n vertices with integers such that the absolute value of the difference between the endpoints of all m of the edges yields a distinct value between 1 and m. A jolly jumper represents a graceful labeling of an n-vertex path. The famous *graceful tree conjecture* asks whether every tree has a graceful labeling. Graceful graphs make a great topic for undergraduate student research. See Gallian's dynamic survey [Gal01] for a list of accessible open problems.

2.5 The mathematics of card shuffling is a fascinating topic. A *perfect shuffle* splits the deck in half into piles A and B, and then interleaves the cards: top of A, top of B, top of A, ... Amazingly, eight perfect shuffles take a deck back to its original state! This can be shown using either modular arithmetic or the theory of cycles in permutations. See [DGK83, Mor98] for more on perfect shuffles.

2.6 The first author of this book has an Erdös number of 2, giving the second a number of ≤ 3. Erdös was famous for posing beautiful, easy-to-understand but hard-to-solve problems in combinatorics, graph theory, and number theory. Read one of the popular biographies of his life [Hof99, Sch00] for more about this fascinating man.

3
Strings

Text strings are a fundamental data structure of growing importance. Internet search engines such as Google search billions of documents almost instantaneously. The sequencing of the human genome has given us three billion characters of text describing all the proteins we are built from. In searching this string for interesting patterns, we are literally looking for the secret of life.

The stakes in solving the programming problems in this chapter are considerably lower than that. However, they provide insight into how characters and text strings are represented in modern computers, and the clever algorithms which search and manipulate this data. We refer the interested reader to [Gus97] for a more advanced discussion of string algorithms.

3.1 Character Codes

Character codes are mappings between numbers and the symbols which make up a particular alphabet. Computers are fundamentally designed to work with numerical data. All they know about a given alphabet is which symbol is assigned to each possible number. When changing the font in a print program, all that really changes are the image bit-maps associated with each character. When changing an operating system from English to Russian all that really changes is the mapping of symbols in the character code.

It is useful to understand something about character code designs when working with text strings. The *American Standard Code for Information Interchange* (ASCII) is a

0	NUL	1	SOH	2	STX	3	ETX	4	EOT	5	ENQ	6	ACK	7	BEL
8	BS	9	HT	10	NL	11	VT	12	NP	13	CR	14	SO	15	SI
16	DLE	17	DC1	18	DC2	19	DC3	20	DC4	21	NAK	22	SYN	23	ETB
24	CAN	25	EM	26	SUB	27	ESC	28	FS	29	GS	30	RS	31	US
32	SP	33	!	34	"	35	#	36	$	37	%	38	&	39	'
40	(	41	)	42	*	43	+	44	,	45	-	46	.	47	/
48	0	49	1	50	2	51	3	52	4	53	5	54	6	55	7
56	8	57	9	58	:	59	;	60	<	61	=	62	>	63	?
64	@	65	A	66	B	67	C	68	D	69	E	70	F	71	G
72	H	73	I	74	J	75	K	76	L	77	M	78	N	79	O
80	P	81	Q	82	R	83	S	84	T	85	U	86	V	87	W
88	X	89	Y	90	Z	91	[	92	/	93	]	94	^	95	_
96	`	97	a	98	b	99	c	100	d	101	e	102	f	103	g
104	h	105	i	106	j	107	k	108	l	109	m	110	n	111	o
112	p	113	q	114	r	115	s	116	t	117	u	118	v	119	w
120	x	121	y	122	z	123	{	124	—	125	}	126	~	127	DEL

Figure 3.1. The ASCII character code.

single-byte character code where $2^7 = 128$ characters are specified.[1] Bytes are eight-bit entities; so that means the highest-order bit is left as zero.

Consider the ASCII character code table presented in Figure 3.1, where the left entry in each pair is the decimal (base-ten) value of the specification, while the right entry is the associated symbol. These symbol assignments were not done at random. Several interesting properties of the design make programming tasks easier:

- All non-printable characters have either the first three bits as zero or all seven lowest bits as one. This makes it very easy to eliminate them before displaying junk, although somehow very few programs seem to do so.
- Both the upper- and lowercase letters and the numerical digits appear sequentially. Thus we can iterate through all the letters/digits simply by looping from the value of the first symbol (say, "`a`") to value of the last symbol (say, "`z`").
- Another consequence of this sequential placement is that we can convert a character (say, "`I`") to its rank in the collating sequence (eighth, if "`A`" is the zeroth character) simply by subtracting off the first symbol ("`A`").
- We can convert a character (say "`C`") from upper- to lowercase by adding the difference of the upper and lowercase starting character ("`C`"-"`A`"+"`a`"). Similarly, a character x is uppercase if and only if it lies between "`A`" and "`Z`".
- Given the character code, we can predict what will happen when naively sorting text files. Which of "`x`" or "`3`" or "`C`" appears first in alphabetical order? Sorting alphabetically means sorting by character code. Using a different collating sequence requires more complicated comparison functions, as will be discussed in Chapter 4.
- Non-printable character codes for new-line (10) and carriage return (13) are designed to delimit the end of text lines. Inconsistent use of these codes is one of the pains in moving text files between UNIX and Windows systems.

More modern international character code designs such as *Unicode* use two or even three bytes per symbol, and can represent virtually any symbol in every language on

[1] Be aware that there are literally dozens of variations on ASCII. Perhaps the most important is ISO Latin-1, which is a full 8-bit code that includes European accented characters.

earth. However, good old ASCII remains alive embedded in Unicode. Whenever the high-order bits are 0, the text gets interpreted as single-byte characters instead of two-byte entities. Thus we can still use the simpler, more memory-efficient encoding while opening the door to thousands of additional symbols.

All of this makes a big difference in manipulating text in different programming languages. Older languages, like Pascal, C, and C++, view the `char` type as virtually synonymous with 8-bit entities. Thus the character data type is the choice for manipulating raw files, even for those which are not intended to be printable. Java, on the other hand, was designed to support Unicode, so characters are 16-bit entities. The upper byte is all zeros when working with ASCII/ISO Latin 1 text. Be aware of this difference when you switch between programming languages.

3.2 Representing Strings

Strings are sequences of characters, where order clearly matters. It is important to be aware of how your favorite programming language represents strings, because there are several different possibilities:

- *Null-terminated Arrays* — C/C++ treats strings as arrays of characters. The string ends the instant it hits the null character "\0", i.e., zero ASCII. Failing to end your string explicitly with a null typically extends it by a bunch of unprintable characters. In defining a string, enough array must be allocated to hold the largest possible string (plus the null) unless you want a core dump. The advantage of this array representation is that all individual characters are accessible by index as array elements.

- *Array Plus Length* — Another scheme uses the first array location to store the length of the string, thus avoiding the need for any terminating null character. Presumably this is what Java implementations do internally, even though the user's view of strings is as objects with a set of operators and methods acting on them.

- *Linked Lists of Characters* — Text strings can be represented using linked lists, but this is typically avoided because of the high space-overhead associated with having a several-byte pointer for each single byte character. Still, such a representation might be useful if you were to insert or delete substrings frequently within the body of a string.

The underlying string representation can have a big impact on which operations are easily or efficiently supported. Compare each of these three data structures with respect to the following properties:

- Which uses the least amount of space? On what sized strings?

- Which constrains the content of the strings which can possibly be represented?

- Which allow constant-time access to the ith character?

- Which allow efficient checks that the ith character is in fact within the string, thus avoiding out-of-bounds errors?
- Which allow efficient deletion or insertion of new characters at the ith position?
- Which representation is used when users are limited to strings of length at most 255, e.g., file names in Windows?

3.3 Program Design Example: Corporate Renamings

Corporate name changes are occurring with ever greater frequency, as companies merge, buy each other out, try to hide from bad publicity, or even raise their stock price – remember when adding a .com to a company's name was the secret to success!

These changes make it difficult to figure out the current name of a company when reading old documents. Your company, Digiscam (formerly Algorist Technologies), has put you to work on a program which maintains a database of corporate name changes and does the appropriate substitutions to bring old documents up to date.

Your program should take as input a file with a given number of corporate name changes, followed by a given number of lines of text for you to correct. Only *exact* matches of the string should be replaced. There will be at most 100 corporate changes, and each line of text is at most 1,000 characters long. A sample input is —

```
4
"Anderson Consulting" to "Accenture"
"Enron" to "Dynegy"
"DEC" to "Compaq"
"TWA" to "American"
5
Anderson Accounting begat Anderson Consulting, which
offered advice to Enron before it DECLARED bankruptcy,
which made Anderson
Consulting quite happy it changed its name
in the first place!
```

Which should be transformed to —

```
Anderson Accounting begat Accenture, which
offered advice to Dynegy before it CompaqLARED bankruptcy,
which made Anderson
Consulting quite happy it changed its name
in the first place!
```

The specifications do not ask you to respect word delimiters (such as blank), so transforming `DECLARED` to `CompaqLARED` is indeed the right thing to do.

Solution starts below

What kind of string operations do we need to do to solve this problem? We must be able to read strings and store them, search strings for patterns, modify them, and finally print them.

Observe that the input file has been segmented into two parts. The first section, the dictionary of name changes, must be completely read and stored before starting to convert the text. To declare the relevant data structures:

```
#include <string.h>

#define MAXLEN          1001    /* longest possible string */
#define MAXCHANGES      101     /* maximum number of name changes */

typedef char string[MAXLEN];

string mergers[MAXCHANGES][2];  /* store before/after corporate names */
int nmergers;                   /* number of different name changes */
```

We represent the dictionary as a two-dimensional array of strings. We do not need to sort the keys in any particular order, since we will be scanning through each of them on each line of text.

Reading the list of company names is somewhat complicated by the fact that we must parse each input line to extract the stuff between quotes. The trick is ignoring text before the first quote, and collecting it until the second quote:

```
read_changes()
{
        int i;                  /* counter */

        scanf("%d\n",&nmergers);
        for (i=0; i<nmergers; i++) {
                read_quoted_string(&(mergers[i][0]));
                read_quoted_string(&(mergers[i][1]));
        }
}

read_quoted_string(char *s)
{
        int i=0;                /* counter */
        char c;                 /* latest character */

        while ((c=getchar()) != '\"') ;
        while ((c=getchar()) != '\"') {
                s[i] = c;
                i = i+1;
        }
        s[i] = '\0';
```

```
}
```

The more advanced operations we will need are presented in the following sections.

3.4 Searching for Patterns

The simplest algorithm to search for the presence of pattern string p in text t overlays the pattern string at every position in the text, and checks whether every pattern character matches the corresponding text character:

```
/*      Return the position of the first occurrence of the pattern p
        in the text t, and -1 if it does not occur.
*/

int findmatch(char *p, char *t)
{
        int i,j;                        /* counters */
        int plen, tlen;                 /* string lengths */

        plen = strlen(p);
        tlen = strlen(t);

        for (i=0; i<=(tlen-plen); i=i+1) {
                j=0;
                while ((j<plen) && (t[i+j]==p[j]))
                        j = j+1;
                if (j == plen) return(i);
        }

        return(-1);
}
```

Note that this routine only searches for exact pattern matches. If a letter is capitalized in the pattern but not in the text there is no match. More seriously, if a company name is split between lines (see the example input), no match will be detected. Such searches can be performed by changing the text/pattern comparison from `t[i+j]==p[j]` to something more interesting. The same technique can be used to allow for *wild card* characters, which match anything. A more general notion of approximate string matching is discussed in Section 11.

This naive algorithm can take as much as $O(|p||t|)$ time in the worst case. Can you construct an arbitrary-length example pattern and text where it actually takes this much time without ever matching the pattern? Usually the naive search will be much more efficient, since we advance in the text soon as we get a single mismatch. More complicated, linear-time search algorithms do exist: see [Gus97] for a complete

discussion. These algorithms are likely implemented in the string library associated with your programming language.

3.5 Manipulating Strings

Manipulating strings requires knowing exactly which string representation you or your language is using. Here, we assume that strings are represented as a sequence of single-byte characters in a null-terminated array, as supported by C.

Treating strings like arrays makes many operations reasonably easy:

- *Computing the Length of a String* — Scan through the characters of the string, adding one to the count each time until we hit the null character.
- *Copying a String* — Unless your programming language supports copying entire arrays in one swoop, you must explicitly loop through each character one at a time. Remember to allocate enough memory for the new copy, and don't forget the null!
- *Reversing a String* — The easiest way to do this is to copy the string from right to left into a second array. The right endpoint is found by computing the string length. Remember to terminate the new string with a null! String reversal can also be done in place by swapping characters if you are willing to destroy the original string.

As an example, we implement a routine to replace a substring at a given position with another string. We will need it for our corporate merger program. The tricky part is moving the other characters in our string to accommodate the new text. If the replacement substring is longer than the original, we must move the suffix out of the way so it isn't stepped on. If the replacement substring is shorter, we must move the suffix in to cover up the hole:

```
/*      Replace the substring of length xlen starting at position pos in
        string s with the contents of string y.
*/

replace_x_with_y(char *s, int pos, int xlen, char *y)
{
        int i;                          /* counter */
        int slen,ylen;                  /* lengths of relevant strings */

        slen = strlen(s);
        ylen = strlen(y);

        if (xlen >= ylen)
                for (i=(pos+xlen); i<=slen; i++) s[i+(ylen-xlen)] = s[i];
        else
```

```
                for (i=slen; i>=(pos+xlen); i--) s[i+(ylen-xlen)] = s[i];

        for (i=0; i<ylen; i++) s[pos+i] = y[i];
}
```

An alternative approach would have been to assemble the new string in a temporary buffer, and then overwrite all the characters of s with the contents of the buffer.

3.6 Completing the Merger

With all of these supporting routines in place, the rest of the program becomes fairly simple:

```
main()
{
        string s;               /* input string */
        char c;                 /* input character */
        int nlines;             /* number of lines in text */
        int i,j;                /* counters */
        int pos;                /* position of pattern in string */

        read_changes();
        scanf("%d\n",&nlines);
        for (i=1; i<=nlines; i=i+1) {         /* read text line */
                j=0;
                while ((c=getchar()) != '\n') {
                        s[j] = c;
                        j = j+1;
                }
                s[j] = '\0';

                for (j=0; j<nmergers; j=j+1)
                        while ((pos=findmatch(mergers[j][0],s)) != -1) {
                           replace_x_with_y(s, pos,
                                strlen(mergers[j][0]), mergers[j][1]);
                        }

                printf("%s\n",s);
        }
}
```

3.7 String Library Functions

Whether you work in C, C++, or Java, be aware of the support provided for characters and strings through libraries or classes. There is no reason to reinvent the wheel.

Strings are not a supported data type of standard Pascal, and so the details vary with the specific implementation.

C Library String Functions

C contains both character and string libraries. The C language *character* library `ctype.h` contains several simple tests and manipulations on character codes. As with all C predicates, true is defined as any non-zero quantity, and false as zero.

```
#include <ctype.h>      /* include the character library */

int isalpha(int c);     /* true if c is either upper or lower case */
int isupper(int c);     /* true if c is upper case */
int islower(int c);     /* true if c is lower case */
int isdigit(int c);     /* true if c is a numerical digit (0-9) */
int ispunct(int c);     /* true if c is a punctuation symbol */
int isxdigit(int c);    /* true if c is a hexadecimal digit (0-9,A-F) */
int isprint(int c);     /* true if c is any printable character */

int toupper(int c);     /* convert c to upper case -- no error checking */
int tolower(int c);     /* convert c to lower case -- no error checking */
```

Check the definition of each carefully before assuming it does exactly what you want.

The following functions appear in the C language *string* library `string.h`. The full library has more functions and options, so check it out.

```
#include <string.h>      /* include the string library */

char *strcat(char *dst, const char *src);     /* concatenation */
int strcmp(const char *s1, const char *s2);   /* is s1 == s2? */
char *strcpy(char *dst, const char *src);     /* copy src to dist */
size_t strlen(const char *s);                 /* length of string */
char *strstr(const char *s1, const char *s2); /* search for s2 in s1 */
char *strtok(char *s1, const char *s2);       /* iterate words in s1 */
```

C++ String Library Functions

In addition to supporting C-style strings, C++ has a string class which contains methods for these operations and more:

```
string::size()          /* string length */
string::empty()         /* is it empty */
```

```
string::c_str()         /* return a pointer to a C style string */

string::operator [](size_type i)        /* access the ith character */

string::append(s)       /* append to string */
string::erase(n,m)      /* delete a run of characters */
string::insert(size_type n, const string&s) /* insert string s at n */

string::find(s)
string::rfind(s)        /* search left or right for the given string */

string::first()
string::last()          /* get characters, also there are iterators */
```

Overloaded operators exist for concatenation and string comparison.

Java String Objects

Java strings are first-class objects deriving either from the `String` class or the `StringBuffer` class. The `String` class is for static strings which do not change, while `StringBuffer` is designed for dynamic strings. Recall that Java was designed to support Unicode, so its characters are 16-bit entities.

The `java.text` package contains more advanced operations on strings, including routines to parse dates and other structured text.

3.8 Problems

3.8.1 WERTYU

PC/UVa IDs: 110301/10082, **Popularity:** A, **Success rate:** high **Level:** 1



A common typing error is to place your hands on the keyboard one row to the right of the correct position. Then "`Q`" is typed as "`W`" and "`J`" is typed as "`K`" and so on. Your task is to decode a message typed in this manner.

Input

Input consists of several lines of text. Each line may contain digits, spaces, uppercase letters (except "`Q`", "`A`", "`Z`"), or punctuation shown above [except back-quote (`)]. Keys labeled with words [`Tab`, `BackSp`, `Control`, etc.] are not represented in the input.

Output

You are to replace each letter or punctuation symbol by the one immediately to its left on the QWERTY keyboard shown above. Spaces in the input should be echoed in the output.

Sample Input

```
O S, GOMR YPFSU/
```

Sample Output

```
I AM FINE TODAY.
```

3.8.2 Where's Waldorf?

PC/UVa IDs: 110302/10010, **Popularity:** B, **Success rate:** average **Level:** 2

Given an m by n grid of letters and a list of words, find the location in the grid at which the word can be found.

A word matches a straight, uninterrupted line of letters in the grid. A word can match the letters in the grid regardless of case (i.e., upper- and lowercase letters are to be treated as the same). The matching can be done in any of the eight horizontal, vertical, or diagonal directions through the grid.

Input

The input begins with a single positive integer on a line by itself indicating the number of cases, followed by a blank line. There is also a blank line between each two consecutive cases.

Each case begins with a pair of integers m followed by n on a single line, where $1 \leq m, n \leq 50$ in decimal notation. The next m lines contain n letters each, representing the grid of letters where the words must be found. The letters in the grid may be in upper- or lowercase. Following the grid of letters, another integer k appears on a line by itself ($1 \leq k \leq 20$). The next k lines of input contain the list of words to search for, one word per line. These words may contain upper- and lowercase letters only – no spaces, hyphens, or other non-alphabetic characters.

Output

For each word in each test case, output a pair of integers representing its location in the corresponding grid. The integers must be separated by a single space. The first integer is the line in the grid where the first letter of the given word can be found (1 represents the topmost line in the grid, and m represents the bottommost line). The second integer is the column in the grid where the first letter of the given word can be found (1 represents the leftmost column in the grid, and n represents the rightmost column in the grid). If a word can be found more than once in the grid, then output the location of the uppermost occurrence of the word (i.e., the occurrence which places the first letter of the word closest to the top of the grid). If two or more words are uppermost, output the leftmost of these occurrences. All words can be found at least once in the grid.

The output of two consecutive cases must be separated by a blank line.

Sample Input

```
1

8 11
abcDEFGhigg
hEbkWalDork
FtyAwaldORm
FtsimrLqsrc
byoArBeDeyv
Klcbqwikomk
strEBGadhrb
yUiqlxcnBjf
4
Waldorf
Bambi
Betty
Dagbert
```

Sample Output

```
2 5
2 3
1 2
7 8
```

3.8.3 Common Permutation

PC/UVa IDs: 110303/10252, **Popularity:** A, **Success rate:** average **Level:** 1

Given two strings a and b, print the longest string x of letters such that there is a permutation of x that is a subsequence of a and there is a permutation of x that is a subsequence of b.

Input

The input file contains several cases, each case consisting of two consecutive lines. This means that lines 1 and 2 are a test case, lines 3 and 4 are another test case, and so on. Each line contains one string of lowercase characters, with first line of a pair denoting a and the second denoting b. Each string consists of at most 1,000 characters.

Output

For each set of input, output a line containing x. If several x satisfy the criteria above, choose the first one in alphabetical order.

Sample Input

```
pretty
women
walking
down
the
street
```

Sample Output

```
e
nw
et
```

3.8.4 Crypt Kicker II

PC/UVa IDs: 110304/850, **Popularity:** A, **Success rate:** average **Level:** 2

A popular but insecure method of encrypting text is to permute the letters of the alphabet. That is, in the text, each letter of the alphabet is consistently replaced by some other letter. To ensure that the encryption is reversible, no two letters are replaced by the same letter.

A powerful method of cryptanalysis is the known plain text attack. In a known plain text attack, the cryptanalyst manages to have a known phrase or sentence encrypted by the enemy, and by observing the encrypted text then deduces the method of encoding.

Your task is to decrypt several encrypted lines of text, assuming that each line uses the same set of replacements, and that one of the lines of input is the encrypted form of the plain text `the quick brown fox jumps over the lazy dog`.

Input

The input begins with a single positive integer on a line by itself indicating the number of test cases, followed by a blank line. There will also be a blank line between each two consecutive cases.

Each case consists of several lines of input, encrypted as described above. The encrypted lines contain only lowercase letters and spaces and do not exceed 80 characters in length. There are at most 100 input lines.

Output

For each test case, decrypt each line and print it to standard output. If there is more than one possible decryption, any one will do. If decryption is impossible, output

```
No solution.
```

The output of each two consecutive cases must be separated by a blank line.

Sample Input

```
1

vtz ud xnm xugm itr pyy jttk gmv xt otgm xt xnm puk ti xnm fprxq
xnm ceuob lrtzv ita hegfd tsmr xnm ypwq ktj
frtjrpgguvj otvxmdxd prm iev prmvx xnmq
```

Sample Output

```
now is the time for all good men to come to the aid of the party
the quick brown fox jumps over the lazy dog
programming contests are fun arent they
```

3.8.5 Automated Judge Script

PC/UVa IDs: 110305/10188, **Popularity:** B, **Success rate:** average **Level:** 1

Human programming contest judges are known to be very picky. To eliminate the need for them, write an automated judge script to judge submitted solution runs.

Your program should take a file containing the correct output as well as the output of submitted program and answer either `Accepted`, `Presentation Error`, or `Wrong Answer`, defined as follows:

Accepted: You are to report "`Accepted`" if the team's output matches the standard solution exactly. *All* characters must match and must occur in the same order.

Presentation Error: Give a "`Presentation Error`" if all *numeric* characters match in the same order, but there is at least one non-matching non-numeric character. For example, "`15 0`" and "`150`" would receive "`Presentation Error`", whereas "`15 0`" and "`1 0`" would receive "`Wrong Answer`," described below.

Wrong Answer: If the team output cannot be classified as above, then you have no alternative but to score the program a '`Wrong Answer`'.

Input

The input will consist of an arbitrary number of input sets. Each input set begins with a line containing a positive integer $n < 100$, which describes the number of lines of the correct solution. The next n lines contain the correct solution. Then comes a positive integer $m < 100$, alone on its line, which describes the number of lines of the team's submitted output. The next m lines contain this output. The input is terminated by a value of $n = 0$, which should not be processed.

No line will have more than 100 characters.

Output

For each set, output one of the following:

`Run #`x`: Accepted`
`Run #`x`: Presentation Error`
`Run #`x`: Wrong Answer`

where x stands for the number of the input set (starting from 1).

Sample Input

```
2
The answer is: 10
The answer is: 5
2
The answer is: 10
```

```
The answer is: 5
2
The answer is: 10
The answer is: 5
2
The answer is: 10
The answer is: 15
2
The answer is: 10
The answer is:  5
2
The answer is: 10
The answer is: 5
3
Input Set #1: YES
Input Set #2: NO
Input Set #3: NO
3
Input Set #0: YES
Input Set #1: NO
Input Set #2: NO
1
1 0 1 0
1
1010
1
The judges are mean!
1
The judges are good!
0
```

Sample Output

```
Run #1: Accepted
Run #2: Wrong Answer
Run #3: Presentation Error
Run #4: Wrong Answer
Run #5: Presentation Error
Run #6: Presentation Error
```

3.8.6 File Fragmentation

PC/UVa IDs: 110306/10132, **Popularity:** C, **Success rate:** average **Level:** 2

Your friend, a biochemistry major, tripped while carrying a tray of computer files through the lab. All of the files fell to the ground and broke. Your friend picked up all the file fragments and called you to ask for help putting them back together again.

Fortunately, all of the files on the tray were identical, all of them broke into exactly two fragments, and all of the file fragments were found. Unfortunately, the files didn't all break in the same place, and the fragments were completely mixed up by their fall to the floor.

The original binary fragments have been translated into strings of ASCII `1`'s and `0`'s. Your job is to write a program that determines the bit pattern the files contained.

Input

The input begins with a single positive integer on its own line indicating the number of test cases, followed by a blank line. There will also be a blank line between each two consecutive cases.

Each case will consist of a sequence of "file fragments," one per line, terminated by the end-of-file marker or a blank line. Each fragment consists of a string of ASCII `1`'s and `0`'s.

Output

For each test case, the output is a single line of ASCII `1`'s and `0`'s giving the bit pattern of the original files. If there are $2N$ fragments in the input, it should be possible to concatenate these fragments together in pairs to make N copies of the output string. If there is no unique solution, any of the possible solutions may be output.

Your friend is certain that there were no more than 144 files on the tray, and that the files were all less than 256 bytes in size.

The output from two consecutive test cases will be separated by a blank line.

Sample Input

```
1

011
0111
01110
111
0111
10111
```

Sample Output

```
01110111
```

3.8.7 Doublets

PC/UVa IDs: 110307/10150, **Popularity:** C, **Success rate:** average **Level:** 3

A *doublet* is a pair of words that differ in exactly one letter; for example, "booster" and "rooster" or "rooster" and "roaster" or "roaster" and "roasted".

You are given a dictionary of up to 25,143 lowercase words, not exceeding 16 letters each. You are then given a number of pairs of words. For each pair of words, find the shortest sequence of words that begins with the first word and ends with the second, such that each pair of adjacent words is a doublet. For example, if you were given the input pair "booster" and "roasted", a possible solution would be ("booster," "rooster," "roaster," "roasted"), provided that these words are all in the dictionary.

Input

The input file contains the dictionary followed by a number of word pairs. The dictionary consists of a number of words, one per line, and is terminated by an empty line. The pairs of input words follow; each pair of words occurs on a line separated by a space.

Output

For each input pair, print a set of lines starting with the first word and ending with the last. Each pair of adjacent lines must be a doublet.

If there are several minimal solutions, any one will do. If there is no solution, print a line: "`No solution.`" Leave a blank line between cases.

Sample Input

```
booster
rooster
roaster
coasted
roasted
coastal
postal

booster roasted
coastal postal
```

Sample Output

```
booster
rooster
roaster
roasted

No solution.
```

3.8.8 Fmt

PC/UVa IDs: 110308/848, **Popularity:** C, **Success rate:** low **Level:** 2

The UNIX program *fmt* reads lines of text, combining and breaking them so as to create an output file with lines as close to 72 characters long as possible without exceeding this limit. The rules for combining and breaking lines are as follows:

- A new line may be started anywhere there is a space in the input. When a new line is started, blanks at the end of the previous line and at the beginning of the new line are eliminated.
- A line break in the input may be eliminated in the output unless (1) it is at the end of a blank or empty line, or (2) it is followed by a space or another line break. When a line break is eliminated, it is replaced by a space.
- Spaces must be removed from the end of each output line.
- Any input word containing more than 72 characters must appear on an output line by itself.

You may assume that the input text does not contain any tabbing characters.

Sample Input

```
   Unix fmt

The unix fmt program reads lines of text, combining
and breaking lines so as to create an
output file with lines as close to without exceeding
72 characters long as possible.  The rules for combining and breaking
lines are as follows.

   1.  A new line may be started anywhere there is a space in the input.
If a new line is started, there will be no trailing blanks at the
end of the previous line or at the beginning of the new line.

   2.  A line break in the input may be eliminated in the output, provided
it is not followed by a space or another line break.  If a line
break is eliminated, it is replaced by a space.
```

Sample Output

```
   Unix fmt

The unix fmt program reads lines of text, combining and breaking lines
so as to create an output file with lines as close to without exceeding
72 characters long as possible.  The rules for combining and breaking
```

lines are as follows.

1. A new line may be started anywhere there is a space in the input. If a new line is started, there will be no trailing blanks at the end of the previous line or at the beginning of the new line.

2. A line break in the input may be eliminated in the output, provided it is not followed by a space or another line break. If a line break is eliminated, it is replaced by a space.

3.9 Hints

3.1 Should you use hard-coded logic to perform the character replacement, or would a table-driven strategy of initialized arrays be easier?

3.2 Can you write a single comparison routine with arguments which can handle comparison in all eight directions when called with the right arguments? Does it pay to specify directions as pairs of integers (δ_x, δ_y) instead of by name?

3.3 Can you rearrange the letters of each word so that the common permutation becomes more apparent?

3.5 What is the easiest way to compare just the numeric characters, as required for identifying presentation errors?

3.6 Can you easily figure out which pairs of fragments go together, if not their order?

3.7 Can we model this problem as a path problem in graphs? It might pay to look ahead to Chapter 9 where we present graph data structures and traversal algorithms.

3.10 Notes

3.4 Although it has a history dating back thousands of years, cryptography has been revolutionized by computational advances and new algorithms. Read Schneier's [Sch94] and/or Stinson's [Sti02] books to learn more about this fascinating area.

3.8 The gold standard among text-formatting programs is *Latex*, the system we used to typeset this book. It is built on top of *TeX*, developed by master computer scientist Don Knuth. He is the author of the famous *Art of Computer Programming* books [Knu73a, Knu81, Knu73b], which are still fascinating and unsurpassed more than 30 years after their original publication.